ಕನ್ವಲ್ಯೂಷನಲ್ ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ಗಳು (CNN)

ಕಂಪ್ಯೂಟರ್‌ಗೆ “ಕಣ್ಣು” ಕೊಡುವ ತಂತ್ರ: Convolution, Pooling, LeNet, VGG, ResNet

ಸರಳ ಕನ್ನಡದಲ್ಲಿ, ದಿನನಿತ್ಯದ ಉದಾಹರಣೆಗಳು ಮತ್ತು ಹಂತ-ಹಂತದ ಲೆಕ್ಕಗಳೊಂದಿಗೆ

“Lecture 7: Convolutional Neural Networks”
ಉಪನ್ಯಾಸ ಸ್ಲೈಡ್‌ಗಳನ್ನು ಆಧರಿಸಿದ ಕನ್ನಡ ಅಧ್ಯಯನ ಮಾರ್ಗದರ್ಶಿ

ಪರಿವಿಡಿ

೧. ಏಕೆ CNN? ಚಿತ್ರಗಳ ಸಮಸ್ಯೆ ಮತ್ತು ImageNet ಕ್ರಾಂತಿ

೨. ಮೆದುಳಿನಿಂದ ಸ್ಫೂರ್ತಿ: Receptive Field ಮತ್ತು ಮೂಲ ತತ್ವಗಳು

೩. Convolution ಎಂದರೇನು? (1D ಉದಾಹರಣೆ)

೪. 2D Convolution: ಫಿಲ್ಟರ್, ಕರ್ನಲ್, ಫೀಚರ್ ಮ್ಯಾಪ್ — ಹಂತ-ಹಂತದ ಲೆಕ್ಕ

೫. ಪ್ರಸಿದ್ಧ ಕರ್ನಲ್‌ಗಳು: Edge, Blur, Sharpen

೬. Zero Padding ಮತ್ತು Stride, ಔಟ್‌ಪುಟ್ ಗಾತ್ರದ ಸೂತ್ರ

೭. ಹಲವು ಫೀಚರ್ ಮ್ಯಾಪ್‌ಗಳು, ಚಾನೆಲ್‌ಗಳು ಮತ್ತು ಪ್ಯಾರಾಮೀಟರ್ ಉಳಿತಾಯ

೮. Pooling ಪದರ

೯. LeNet-5 ಮತ್ತು Conv ಪದರದ ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು

೧೦. MNIST ಅಂಕೆ ಗುರುತಿಸುವಿಕೆ — CNN ಕೋಡ್ ಮತ್ತು ಪ್ಯಾರಾಮೀಟರ್ ಲೆಕ್ಕ

೧೧. ಪಠ್ಯಕ್ಕೆ CNN: IMDB ವಿಮರ್ಶೆಗಳು (Conv1D)

೧೨. ಪ್ರಸಿದ್ಧ ವಿನ್ಯಾಸಗಳು: AlexNet, VGG16, ResNet

೧೩. ಸಾರಾಂಶ, ಪದಕೋಶ ಮತ್ತು ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

೧. ಏಕೆ CNN? ಚಿತ್ರಗಳ ಸಮಸ್ಯೆ ಮತ್ತು ImageNet ಕ್ರಾಂತಿ

ಇದುವರೆಗೆ ಬಳಸಿದ Dense (fully connected) ನೆಟ್‌ವರ್ಕ್‌ನಲ್ಲಿ ಪ್ರತಿ ನ್ಯೂರಾನ್ ಚಿತ್ರದ **ಎಲ್ಲ ಪಿಕ್ಸೆಲ್‌ಗಳಿಗೂ** ಸಂಪರ್ಕ ಹೊಂದಿರುತ್ತದೆ. ಇದು ಚಿತ್ರಗಳಿಗೆ ಎರಡು ದೊಡ್ಡ ಸಮಸ್ಯೆ ತರುತ್ತದೆ:

1. **ಪ್ಯಾರಾಮೀಟರ್‌ಗಳ ಸ್ಫೋಟ.** ಒಂದು ಸಾಮಾನ್ಯ ಬಣ್ಣದ ಚಿತ್ರ 224×224×3 = 150,528 ಸಂಖ್ಯೆಗಳು.
2. **ಪಕ್ಕದ ಪಿಕ್ಸೆಲ್‌ಗಳ ಸಂಬಂಧ ಗೊತ್ತಿಲ್ಲ.** Dense ಪದರಕ್ಕೆ ಪಿಕ್ಸೆಲ್‌ಗಳ ಕ್ರಮ ಅಥವಾ ಸ್ಥಾನ ಮುಖ್ಯವಲ್ಲ — ಚಿತ್ರದ ಪಿಕ್ಸೆಲ್‌ಗಳನ್ನು ಕಲಸಿದರೂ (ಎಲ್ಲ ಚಿತ್ರಗಳಲ್ಲಿ ಒಂದೇ ರೀತಿ) ಅದಕ್ಕೆ ವ್ಯತ್ಯಾಸ ತಿಳಿಯುವುದಿಲ್ಲ. ಆದರೆ ಚಿತ್ರದಲ್ಲಿ ಹತ್ತಿರದ ಪಿಕ್ಸೆಲ್‌ಗಳು ಒಟ್ಟಿಗೆ ಅರ್ಥ (ಗೆರೆ, ಅಂಚು, ಆಕಾರ) ಕೊಡುತ್ತವೆ.

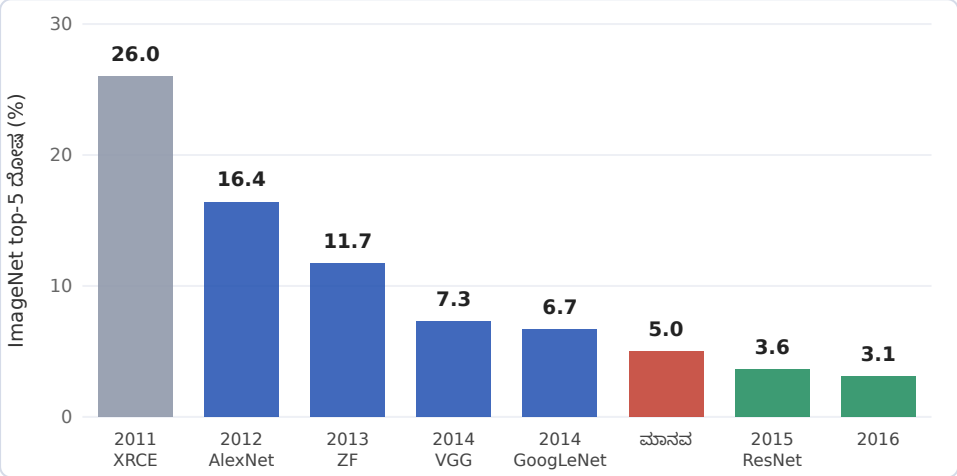
> **ಲೆಕ್ಕ: Dense vs Convolution — ಪ್ಯಾರಾಮೀಟರ್ ಹೋಲಿಕೆ**
>
> 224×224×3 ಚಿತ್ರದ ಮೇಲೆ 64 ಲಕ್ಷಣಗಳನ್ನು ಕಲಿಯಬೇಕು:
>
> ```
> Dense (64 ನ್ಯೂರಾನ್)        : 150,528 × 64 + 64 = 9,633,856   (ಸುಮಾರು 96 ಲಕ್ಷ!)
> Conv (64 ಫಿಲ್ಟರ್, 3×3)      : 64 × (3×3×3 + 1)  =     1,792   (ಕೇವಲ ಸುಮಾರು ಎರಡು ಸಾವಿರ)
> ```
>
> ಅಂದರೆ ಸುಮಾರು **5,400 ಪಟ್ಟು ಕಡಿಮೆ** ಪ್ಯಾರಾಮೀಟರ್! ಇದು ಹೇಗೆ ಸಾಧ್ಯ — ಒಂದು ಚಿಕ್ಕ ಫಿಲ್ಟರ್ ಅನ್ನೇ ಇಡೀ ಚಿತ್ರದ ಮೇಲೆ ಜಾರಿಸಿ ಮತ್ತೆ ಮತ್ತೆ ಬಳಸುವುದರಿಂದ (weight sharing). ಇದೇ CNN ನ ರಹಸ್ಯ.

CNN ಬಳಕೆಗಳು

ಚಿತ್ರ ಹುಡುಕಾಟ (image search), ವೀಡಿಯೊ ವರ್ಗೀಕರಣ, ಸ್ವಯಂಚಾಲಿತ ಕಾರುಗಳು, ಧ್ವನಿ ಗುರುತಿಸುವಿಕೆ, ಭಾಷಾ ಸಂಸ್ಕರಣೆ (NLP), ವೈದ್ಯಕೀಯ ಚಿತ್ರಣ (X-ray, MRI), ಮುಖ ಗುರುತಿಸುವಿಕೆ.

2012: ಕ್ರಾಂತಿ — AlexNet

ImageNet LSVRC ಚಿತ್ರ ಗುರುತಿಸುವ ಸ್ಪರ್ಧೆಯಲ್ಲಿ 2012 ರಲ್ಲಿ ಅಲೆಕ್ಸ್ ಕ್ರಿಝೆವ್ಸ್ಕಿ (Alex Krizhevsky) ಅವರ **AlexNet** (CNN) ಹಿಂದಿನ ವರ್ಷದ ದೋಷ ಪ್ರಮಾಣವನ್ನು 26.0% ರಿಂದ 16.4% ಕ್ಕೆ ಇಳಿಸಿತು. ನಂತರದ ವರ್ಷಗಳಲ್ಲಿ CNN ಗಳು ಇನ್ನಷ್ಟು ಸುಧಾರಿಸಿ ಮಾನವನ ಮಟ್ಟವನ್ನೂ (5.0%) ಮೀರಿಸಿದವು — ಹಾಗಾಗಿ ಸ್ಲೈಡ್‌ನ ಶೀರ್ಷಿಕೆ: “AI has overtaken man!!!”.



ಚಿತ್ರ ೧: ImageNet top-5 ದೋಷ ಪ್ರಮಾಣ (ಸ್ಲೈಡ್ 2 ರ ಗ್ರಾಫ್‌ನ ಸಂಖ್ಯೆಗಳು). ಕಡಿಮೆ = ಉತ್ತಮ.

೨. ಮೆದುಳಿನಿಂದ ಸ್ಫೂರ್ತಿ: Receptive Field ಮತ್ತು ಮೂಲ ತತ್ವಗಳು

ಮೆದುಳಿನ ದೃಶ್ಯ ಕೇಂದ್ರ

ಡೇವಿಡ್ ಹ್ಯೂಬೆಲ್ (David Hubel) ಮತ್ತು ಟೋರ್ಸ್ಟನ್ ವೀಸೆಲ್ (Torsten Wiesel) ಮೆದುಳಿನ **visual cortex** ಮಾದರಿಗಳನ್ನು ಹೇಗೆ ಸಂಸ್ಕರಿಸುತ್ತದೆ ಎಂದು ಕಂಡುಹಿಡಿದರು ಮತ್ತು 1981 ರಲ್ಲಿ ನೊಬೆಲ್ ಪ್ರಶಸ್ತಿ ಪಡೆದರು. ಅವರ ಆವಿಷ್ಕಾರಗಳು:

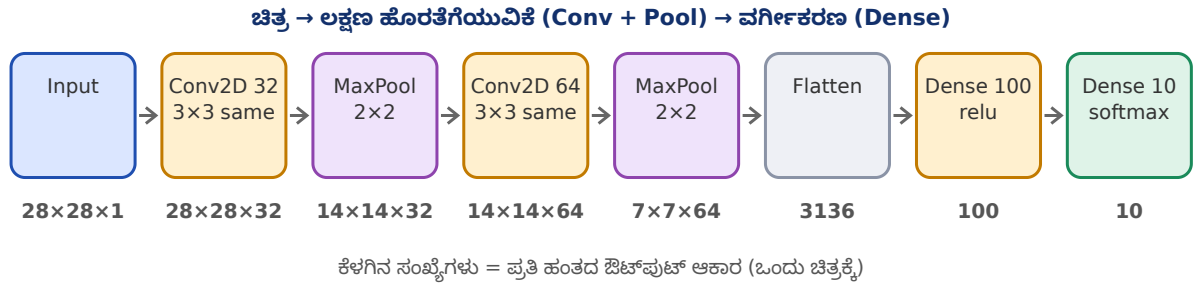
- ಮೆದುಳಿನ ಅನೇಕ ನ್ಯೂರಾನ್‌ಗಳು ದೃಶ್ಯದ ಒಂದು **ಚಿಕ್ಕ ಸ್ಥಳೀಯ ಭಾಗವನ್ನು** ಮಾತ್ರ ನೋಡುತ್ತವೆ — ಇದೇ **Local Receptive Field (LRF)**. ಬೇರೆ ನ್ಯೂರಾನ್‌ಗಳ ಕ್ಷೇತ್ರಗಳು ಅತಿಕ್ರಮಿಸಿ ಇಡೀ ದೃಶ್ಯವನ್ನು ಆವರಿಸುತ್ತವೆ.
- ಕೆಲವು ನ್ಯೂರಾನ್‌ಗಳು ನಿರ್ದಿಷ್ಟ ದಿಕ್ಕಿನ ಗೆರೆಗಳಿಗೆ ಮಾತ್ರ (ಅಡ್ಡ ಅಥವಾ ನಿಂತ) ಪ್ರತಿಕ್ರಿಯಿಸುತ್ತವೆ.
- ಇನ್ನು ಕೆಲವು ದೊಡ್ಡ receptive field ಉಳ್ಳ ನ್ಯೂರಾನ್‌ಗಳು ಕೆಳ ಹಂತದ ನ್ಯೂರಾನ್‌ಗಳ ಫಲಿತಾಂಶ ಕೂಡಿಸಿ **ಸಂಕೀರ್ಣ ಮಾದರಿ**ಗಳಿಗೆ ಪ್ರತಿಕ್ರಿಯಿಸುತ್ತವೆ → **ಶ್ರೇಣೀಕೃತ (hierarchical) ರಚನೆ**.

> **ಹೋಲಿಕೆ: ಅಕ್ಷರ → ಪದ → ವಾಕ್ಯ**
> ಕನ್ನಡ ಓದುವಾಗ ಮೊದಲು ಅಕ್ಷರದ ಗೆರೆ-ವಕ್ರಗಳನ್ನು ನೋಡುತ್ತೇವೆ, ಅವುಗಳಿಂದ ಅಕ್ಷರ, ಅಕ್ಷರಗಳಿಂದ ಪದ, ಪದಗಳಿಂದ ವಾಕ್ಯದ ಅರ್ಥ. ಹಾಗೆಯೇ ಮುಖ ಗುರುತಿಸುವಾಗ: **ಗೆರೆಗಳು/ಅಂಚುಗಳು → ಕಣ್ಣು-ಮೂಗು-ಬಾಯಿ → ಇಡೀ ಮುಖ**. CNN ನ ಮೊದಲ ಪದರಗಳು ಸರಳ ಅಂಚುಗಳನ್ನು, ಆಳದ ಪದರಗಳು ಸಂಕೀರ್ಣ ಆಕಾರಗಳನ್ನು ಕಲಿಯುತ್ತವೆ.

ಈ ಜೈವಿಕ ಮಾದರಿಯಿಂದ 1980 ರಲ್ಲಿ **Neocognitron** ಬಂತು; ಅದೇ ಮುಂದೆ CNN ಆಗಿ ಬೆಳೆಯಿತು. **LeNet** (ಯಾನ್ ಲೆಕನ್, Yann LeCun) ಮೊದಲ ಪ್ರಾಯೋಗಿಕ CNN: ಏಕದಿಕ್ಕಿನ (feedback ಇಲ್ಲ) ಬಹು-ಪದರ ರಚನೆ, convolution ಮತ್ತು subsampling (pooling) ಪದರಗಳು, ಸಾಮಾನ್ಯ backpropagation ತರಬೇತಿ, ಸಾಮಾನ್ಯ activation ಫಂಕ್ಷನ್‌ಗಳು. (ಸ್ಲೈಡ್ 1988 ಎಂದಿದೆ; LeNet-5 ಕುರಿತ ಪ್ರಸಿದ್ಧ ಪ್ರಕಟಿತ ಪೇಪರ್ ಲೆಕನ್, ಬೊತೂ, ಹ್ಯಾಫ್ನರ್, ಬೆಂಜಿಯೋ ಅವರದು — ಸ್ಲೈಡ್ ಚಿತ್ರದಲ್ಲಿ ತೋರಿಸಿದ ಪೇಪರ್ ಸುಮಾರು 1998 ರದು.)

CNN ನ ಮೂಲ ತತ್ವಗಳು

> **ಕನ್ವಲ್ಯೂಷನ್ ಪದರದ ಮುಖ್ಯ ಉದ್ದೇಶ: ಚಿತ್ರದಿಂದ ಉಪಯುಕ್ತ ಲಕ್ಷಣಗಳನ್ನು (features) ಹೊರತೆಗೆಯುವುದು.**
> **ಎರಡು ಮುಖ್ಯ ಲಕ್ಷಣಗಳು:** (೧) **ಹಂಚಿಕೆಯ ತೂಕಗಳು (shared weights)** — ಒಂದೇ ಫಿಲ್ಟರ್ ಇಡೀ ಚಿತ್ರದ ಮೇಲೆ; (೨) **ಸ್ಥಳೀಯ ಸಂಬಂಧ** — ಹತ್ತಿರದ ಪಿಕ್ಸೆಲ್‌ಗಳು ಸಂಬಂಧಿತ ಮಾಹಿತಿ ಹೊಂದಿರುತ್ತವೆ.
> **ಸಾಮಾನ್ಯ ಪದರಗಳು:** Convolutional layer → Pooling layer (max/average) → Fully connected layer.



ಚಿತ್ರ ೨: ಸ್ಲೈಡ್‌ನ MNIST CNN (ಭಾಗ ೧೦) ನ ಸಂಪೂರ್ಣ ಹರಿವು — ಪ್ರತಿ ಹಂತದ ಆಕಾರ ಕೆಳಗೆ

೩. Convolution ಎಂದರೇನು? (1D ಉದಾಹರಣೆ)

ಗಣಿತದಲ್ಲಿ convolution ಎಂದರೆ ಎರಡು ಫಂಕ್ಷನ್‌ಗಳಿಂದ ಮೂರನೆಯ ಫಂಕ್ಷನ್ ಮಾಡುವ ಕ್ರಿಯೆ — ಒಂದು ಇನ್ನೊಂದರ ಮೇಲೆ ಎಷ್ಟು ಅತಿಕ್ರಮಿಸುತ್ತದೆ ಎಂದು ತೋರಿಸುತ್ತದೆ; signal processing ನಲ್ಲಿ ಬಹಳ ಮುಖ್ಯ. ನಮಗೆ ಬೇಕಾದ ರೂಪ ಸರಳ: **ಕರ್ನಲ್ ಅನ್ನು ಇನ್‌ಪುಟ್ ಮೇಲೆ ಜಾರಿಸುತ್ತಾ, ಪ್ರತಿ ಸ್ಥಾನದಲ್ಲಿ ಗುಣಿಸಿ ಕೂಡಿಸುವುದು (dot product)**.

ಪದ	ಅರ್ಥ
Kernel / Filter	convolution ನಲ್ಲಿ ಬಳಸುವ ತೂಕಗಳ ಚಿಕ್ಕ ಗುಂಪು (ನ್ಯೂರಾನ್‌ನ ತೂಕಗಳನ್ನು ಚಿಕ್ಕ ಚಿತ್ರದಂತೆ ನೋಡಬಹುದು)
Convolution window	ಇನ್‌ಪುಟ್ ಮೇಲೆ ಜಾರುವ ಚಿಕ್ಕ ತುಂಡು
Receptive field	ಒಂದು ನ್ಯೂರಾನ್ ಪ್ರತಿಕ್ರಿಯಿಸುವ ಇನ್‌ಪುಟ್ ಪ್ರದೇಶ
Feature map	ಫಿಲ್ಟರ್ ಅನ್ನು ಇನ್‌ಪುಟ್‌ಗೆ ಅನ್ವಯಿಸಿದ ಫಲಿತಾಂಶ

ಸ್ಲೈಡ್‌ನ ಉದಾಹರಣೆ: 1D ಸಂಕೇತ

ಸಂಕೇತ: [1, 2, 4, 5, 77, 11, 109, ...]. ಕರ್ನಲ್: ಪ್ರತಿ ತೂಕ 0.2. ಮೊದಲ ಕಿಟಕಿ [2, 4, 5] ಗೆ ಸ್ಲೈಡ್ ತೋರಿಸಿದೆ: `A = 2×0.2 + 4×0.2 + 5×0.2 = 2.2` . (ಸ್ಲೈಡ್‌ನ ಚಿತ್ರದಲ್ಲಿ ಕರ್ನಲ್‌ಗೆ ನಾಲ್ಕು 0.2 ಬರೆದಿದೆ ಆದರೆ ಲೆಕ್ಕ ಮೂರು ಪದಗಳದು; ನಾವು 3 ತೂಕಗಳ ಕರ್ನಲ್ ತೆಗೆದುಕೊಳ್ಳೋಣ.) ಕಿಟಕಿಯನ್ನು ಒಂದೊಂದಾಗಿ ಜಾರಿಸಿದರೆ:

ಕಿಟಕಿ	ಲೆಕ್ಕ (0.2 × ಮೊತ್ತ)	ಔಟ್‌ಪುಟ್
[1, 2, 4]	0.2 × 7	1.4
[2, 4, 5]	0.2 × 11	**2.2**
[4, 5, 77]	0.2 × 86	17.2
[5, 77, 11]	0.2 × 93	18.6
[77, 11, 109]	0.2 × 197	39.4

ಏಳು ಸಂಖ್ಯೆಗಳ ಇನ್‌ಪುಟ್ → ಐದು ಔಟ್‌ಪುಟ್ (7 − 3 + 1 = 5). ಈ ಕರ್ನಲ್ ಒಂದು "ತೂಕದ ಚಲಿಸುವ ಸರಾಸರಿ" (moving average)ಯಂತಿದೆ. *ನಿಜ ಜೀವನದ ಹೋಲಿಕೆ:* ಮೂರು ದಿನಗಳ ಮಳೆ ಅಂಕಿಗಳನ್ನು ಕೂಡಿ ಸರಾಸರಿ ತೆಗೆದು ಗ್ರಾಫ್ ಅನ್ನು ನಯಗೊಳಿಸುವಂತೆ.

ಇದೇ convolution ನ ಸಾರ: ಕರ್ನಲ್ ಅನ್ನು ಇನ್‌ಪುಟ್ ಮೇಲೆ ಜಾರಿಸಿ, ಹೊಂದುವ ಸ್ಥಾನಗಳನ್ನು ಗುಣಿಸಿ, ಒಂದು ಹೆಜ್ಜೆಗೆ ಒಂದು ಔಟ್‌ಪುಟ್ ಮೌಲ್ಯ ಪಡೆಯುವುದು.

೪. 2D Convolution: ಫಿಲ್ಟರ್, ಕರ್ನಲ್, ಫೀಚರ್ ಮ್ಯಾಪ್ — ಹಂತ-ಹಂತದ ಲೆಕ್ಕ

ಚಿತ್ರಗಳು 2D ಆದ್ದರಿಂದ ಕರ್ನಲ್ ಕೂಡ 2D (ಸಾಮಾನ್ಯವಾಗಿ **3×3 ಅಥವಾ 5×5**; 7×7, 9×9 ಅಪರೂಪ). ಇದನ್ನು ಇನ್‌ಪುಟ್ ಚಿತ್ರದ ಮೇಲೆ ಎಡದಿಂದ ಬಲಕ್ಕೆ, ಮೇಲಿನಿಂದ ಕೆಳಕ್ಕೆ ಜಾರಿಸುತ್ತೇವೆ. ಪ್ರತಿ ಸ್ಥಾನದಲ್ಲಿ 3×3 ತುಂಡು ಮತ್ತು ಕರ್ನಲ್‌ನ ಅಂಶ-ವಾರು ಗುಣಲಬ್ಧಗಳ ಮೊತ್ತ ಒಂದು ಔಟ್‌ಪುಟ್ ಸಂಖ್ಯೆ.

> **ಹೋಲಿಕೆ: ರಂಗೋಲಿ ಮಾದರಿ ಹುಡುಕುವುದು**
>
> ನಿಮ್ಮ ಬಳಿ ಒಂದು ಚಿಕ್ಕ ಪಾರದರ್ಶಕ ಟೆಂಪ್ಲೇಟ್ (ಕರ್ನಲ್) ಇದೆ ಎಂದುಕೊಳ್ಳಿ. ಅದನ್ನು ದೊಡ್ಡ ರಂಗೋಲಿಯ ಮೇಲೆ ಜಾರಿಸುತ್ತಾ ಪ್ರತಿ ಸ್ಥಳದಲ್ಲೂ "ಈ ಜಾಗದಲ್ಲಿ ನನ್ನ ಮಾದರಿ ಎಷ್ಟು ಹೊಂದುತ್ತದೆ?" ಎಂದು ಅಂಕ ಕೊಡುತ್ತೀರಿ. ಹೆಚ್ಚು ಅಂಕ ಸಿಕ್ಕ ಸ್ಥಳಗಳು = ಆ ಮಾದರಿ ಇರುವ ಜಾಗಗಳು. ಅಂಕಗಳ ಇಡೀ ನಕ್ಷೆಯೇ ಫೀಚರ್ ಮ್ಯಾಪ್.

ಸ್ಲೈಡ್‌ನ ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆ (7 ನಿಮಿಷ): I (6×6) ಮತ್ತು K (3×3), stride = 1

ಇನ್‌ಪುಟ್ I (6×6)

1	2	3	1	2	3
-1	0	3	0	1	2
1	2	-3	1	-1	0
-1	2	3	0	-1	3
1	0	-3	1	3	-2
-2	1	0	1	2	1

ಕರ್ನಲ್ K (3×3)

0	1	0
1	0	1
1	0	1

→

ಫೀಚರ್ ಮ್ಯಾಪ್ (4×4)

2	6	1	5
0	8	-2	5
2	0	3	1
-2	6	2	0

ಚಿತ್ರ ೩: ಕೆಂಪು 3×3 ಕಿಟಕಿ ಮೇಲಿನ ಎಡ ಮೂಲೆಯಲ್ಲಿದ್ದಾಗ ಫಲಿತಾಂಶ = 2 (ಔಟ್‌ಪುಟ್‌ನ ಮೊದಲ ಸೆಲ್)

ಸೆಲ್ (0, 0): ಮೇಲಿನ ಎಡ ಕಿಟಕಿ

ಕಿಟಕಿ = [[1, 2, 3], [−1, 0, 3], [1, 2, −3]], ಕರ್ನಲ್ K = [[0, 1, 0], [1, 0, 1], [1, 0, 1]]

```
ಸಾಲು 1: 1×0 + 2×1 + 3×0     = 0 + 2 + 0   =  2
ಸಾಲು 2: −1×1 + 0×0 + 3×1    = −1 + 0 + 3  =  2
ಸಾಲು 3: 1×1 + 2×0 + (−3)×1  = 1 + 0 − 3   = −2
ಮೊತ್ತ = 2 + 2 − 2 = 2        → ಔಟ್‌ಪುಟ್[0][0] = 2
```

ಸೆಲ್ (0, 1): ಕಿಟಕಿಯನ್ನು ಒಂದು ಕಾಲಮ್ ಬಲಕ್ಕೆ ಸರಿಸಿ

ಕಿಟಕಿ = [[2, 3, 1], [0, 3, 0], [2, −3, 1]]

```
ಸಾಲು 1: 2×0 + 3×1 + 1×0   = 3
ಸಾಲು 2: 0×1 + 3×0 + 0×1   = 0
ಸಾಲು 3: 2×1 + (−3)×0 + 1×1 = 3
ಮೊತ್ತ = 6                   → ಔಟ್‌ಪುಟ್[0][1] = 6
```

ಇದೇ ರೀತಿ ಎಲ್ಲ 16 ಸ್ಥಾನಗಳಿಗೆ ಮಾಡಿದರೆ **ಪೂರ್ಣ ಉತ್ತರ (4×4)**:

2	6	1	5
0	8	−2	5
2	0	3	1
−2	6	2	0

ಔಟ್‌ಪುಟ್ ಗಾತ್ರ: 6 − 3 + 1 = 4 (ಪ್ಯಾಡಿಂಗ್ ಇಲ್ಲದಿದ್ದರೆ ಇನ್‌ಪುಟ್‌ಗಿಂತ ಚಿಕ್ಕದು).

ತಾಂತ್ರಿಕ ಟಿಪ್ಪಣಿ (ನನ್ನ ಸೇರ್ಪಡೆ)

ಗಣಿತದ ನಿಜವಾದ convolution ನಲ್ಲಿ ಕರ್ನಲ್ ಅನ್ನು ಮೊದಲು 180° ತಿರುಗಿಸುತ್ತಾರೆ. ಆದರೆ ಸ್ಲೈಡ್ ಮತ್ತು Keras ನಲ್ಲಿ (ಮತ್ತು ಇಲ್ಲಿ) ತಿರುಗಿಸದೆ ಕೇವಲ “ಜಾರಿಸಿ dot product” ಮಾಡುತ್ತೇವೆ (ತಾಂತ್ರಿಕವಾಗಿ ಇದು cross-correlation). ಕರ್ನಲ್ ಅನ್ನು ನೆಟ್‌ವರ್ಕ್ ಕಲಿಯುವುದರಿಂದ ಈ ವ್ಯತ್ಯಾಸ ಮುಖ್ಯವಲ್ಲ.

ಕೋಡ್‌ನಲ್ಲಿ ಪರೀಕ್ಷಿಸಿ (ನನ್ನ ಸೇರ್ಪಡೆ — Python + NumPy)

```
import numpy as np

I = np.array([[ 1, 2, 3, 1, 2, 3],
              [-1, 0, 3, 0, 1, 2],
              [ 1, 2,-3, 1,-1, 0],
              [-1, 2, 3, 0,-1, 3],
              [ 1, 0,-3, 1, 3,-2],
              [-2, 1, 0, 1, 2, 1]])
K = np.array([[0, 1, 0],
              [1, 0, 1],
              [1, 0, 1]])

def conv2d(M, K, stride=1):
    kh, kw = K.shape
    out_h = (M.shape[0] - kh) // stride + 1      # ಔಟ್‌ಪುಟ್ ಎತ್ತರ
    out_w = (M.shape[1] - kw) // stride + 1      # ಔಟ್‌ಪುಟ್ ಅಗಲ
    out = np.zeros((out_h, out_w), dtype=int)
    for i in range(out_h):
        for j in range(out_w):
            window = M[i*stride:i*stride+kh, j*stride:j*stride+kw]   # ಕಿಟಕಿ ಕತ್ತರಿಸು
            out[i, j] = (window * K).sum()                           # ಗುಣಿಸಿ ಕೂಡಿಸು
    return out

print(conv2d(I, K))      # [[ 2  6  1  5] [ 0  8 -2  5] [ 2  0  3  1] [-2  6  2  0]]
```

೫. ಪ್ರಸಿದ್ಧ ಕರ್ನಲ್‌ಗಳು: Edge, Blur, Sharpen

ಕರ್ನಲ್ ಸಂಖ್ಯೆಗಳು ಏನು ಮಾಡುತ್ತವೆ ಎಂದು ತಿಳಿಯಲು ಕೆಲವು ಹಳೆಯ ಹಸ್ತ-ವಿನ್ಯಾಸದ ಕರ್ನಲ್‌ಗಳನ್ನು ನೋಡೋಣ. **CNN ನಲ್ಲಿ ಇಂತಹ ಕರ್ನಲ್‌ಗಳನ್ನು ನಾವು ಬರೆಯುವುದಿಲ್ಲ — ನೆಟ್‌ವರ್ಕ್ ತರಬೇತಿಯಲ್ಲಿ ತಾನಾಗಿ ಕಲಿಯುತ್ತದೆ.** ಬೇರೆ ಬೇರೆ ಕರ್ನಲ್‌ಗಳು ಬೇರೆ ಬೇರೆ ಲಕ್ಷಣ ಹಿಡಿಯುತ್ತವೆ.

ಕರ್ನಲ್	ಸಂಖ್ಯೆಗಳು	ಕೆಲಸ
ಲಂಬ ಅಂಚು ಪತ್ತೆ (Sobel)	[−1 0 1; −2 0 2; −1 0 1]	ಎಡ-ಬಲ ಪಿಕ್ಸೆಲ್ ವ್ಯತ್ಯಾಸ ಹೆಚ್ಚಿರುವಲ್ಲಿ ದೊಡ್ಡ ಮೌಲ್ಯ → ನಿಂತ ಗೆರೆಗಳು
Blur (ಮಸುಕು)	ಎಲ್ಲ ಒಂಬತ್ತೂ 1/9	ಪಕ್ಕದ ಪಿಕ್ಸೆಲ್‌ಗಳ ಸರಾಸರಿ → ಚಿತ್ರ ನಯ
Edge separation (Laplacian)	[0 −1 0; −1 4 −1; 0 −1 0]	ಎಲ್ಲ ದಿಕ್ಕಿನ ಗಡಿ/ಪರಿವರ್ತನೆ ಒತ್ತಿ ತೋರಿಸುತ್ತದೆ
Sharpen (ತೀಕ್ಷ್ಣ)	[0 −1 0; −1 5 −1; 0 −1 0]	ಸೂಕ್ಷ್ಮ ವಿವರ ಮತ್ತು ಅಂಚು ವರ್ಧಿಸುತ್ತದೆ

ಸ್ಲೈಡ್‌ನಲ್ಲಿ ಕಂಡ ಒಂದು ತಪ್ಪು

ಸ್ಲೈಡ್ 11 ರಲ್ಲಿ "Sharpening" ಕರ್ನಲ್ ಆಗಿ [−1 0 1; −2 0 2; −1 0 1] ತೋರಿಸಿದೆ — ಇದು ಮೊದಲ ಕರ್ನಲ್‌ನಂತೆಯೇ (Sobel ಅಂಚು ಪತ್ತೆ) ಇದೆ. ನಿಜವಾದ ತೀಕ್ಷ್ಣಗೊಳಿಸುವ ಕರ್ನಲ್ ಸಾಮಾನ್ಯವಾಗಿ [0 −1 0; −1 5 −1; 0 −1 0]. ಮೇಲಿನ ಕೋಷ್ಟಕದಲ್ಲಿ ಸರಿಯಾದ ರೂಪ ಬಳಸಿದ್ದೇವೆ.

ಲೆಕ್ಕ ೧: ಲಂಬ ಅಂಚು ಪತ್ತೆ

5×5 ಚಿತ್ರ — ಪ್ರತಿ ಸಾಲು [0, 0, 0, 9, 9] (ಎಡಭಾಗ ಕಪ್ಪು, ಬಲಭಾಗ ಪ್ರಕಾಶಮಾನ; ಲಂಬ ಅಂಚು 3 ಮತ್ತು 4 ನೇ ಕಾಲಮ್ ನಡುವೆ). Sobel ಕರ್ನಲ್ ಅನ್ವಯಿಸಿದಾಗ (ಪ್ಯಾಡಿಂಗ್ ಇಲ್ಲ → 3×3):

```
ಕಿಟಕಿ ಕಾಲಮ್‌ಗಳು 0–2: ಎಲ್ಲವೂ 0                          → ಔಟ್‌ಪುಟ್ 0   (ಅಂಚು ಇಲ್ಲ)
ಕಿಟಕಿ ಕಾಲಮ್‌ಗಳು 1–3: [0, 0, 9] ಪ್ರತಿ ಸಾಲು
   ಪ್ರತಿ ಸಾಲಿನ ಗುಣಲಬ್ಧ: 9×(+1) = 9 → ತೂಕ ಸಾಲುಗಳು 1,2,1 → 9×1 + 9×2 + 9×1 = 36
ಕಿಟಕಿ ಕಾಲಮ್‌ಗಳು 2–4: [0, 9, 9] ಪ್ರತಿ ಸಾಲು → ಇದೂ 36
ಒಟ್ಟು ಔಟ್‌ಪುಟ್ (3×3): ಪ್ರತಿ ಸಾಲು [0, 36, 36]
```

ಅಂಚು ಇರುವ ಸ್ಥಳಗಳಲ್ಲಿ ದೊಡ್ಡ ಮೌಲ್ಯ (36), ಸಮತಟ್ಟಾದ ಸ್ಥಳದಲ್ಲಿ 0.

ಲೆಕ್ಕ ೨: Blur

ಪ್ರತಿ ಸಾಲು [9, 9, 9, 0, 0] (ಹಠಾತ್ ಅಂಚು). 3×3 blur (ಎಲ್ಲ 1/9):

```
ಕಿಟಕಿ ಕಾಲಮ್‌ಗಳು 0–2: [9,9,9]  → ಸರಾಸರಿ 9
ಕಿಟಕಿ ಕಾಲಮ್‌ಗಳು 1–3: [9,9,0]  → ಸರಾಸರಿ 6
ಕಿಟಕಿ ಕಾಲಮ್‌ಗಳು 2–4: [9,0,0]  → ಸರಾಸರಿ 3        ಔಟ್‌ಪುಟ್ ಸಾಲು: [9, 6, 3]
```

ಹಠಾತ್ ಜಿಗಿತ 9 → 0 ಈಗ ಮೃದು ಇಳಿಜಾರು 9 → 6 → 3 ಆಯಿತು (ಮಸುಕು).

ಸ್ಲೈಡ್ 13: ಗೆರೆಗಳನ್ನು ಹೈಲೈಟ್ ಮಾಡುವುದು

ಸ್ಲೈಡ್‌ನಲ್ಲಿ ಇನ್‌ಪುಟ್ ದೇವಾಲಯದ ಚಿತ್ರಕ್ಕೆ ಎರಡು ಫಿಲ್ಟರ್ ಹಾಕಲಾಗಿದೆ — **ಲಂಬ ಫಿಲ್ಟರ್** (7×7 ಸೊನ್ನೆ ಮ್ಯಾಟ್ರಿಕ್ಸ್‌ನ ಮಧ್ಯದ ಕಾಲಮ್ = 1) ಲಂಬ ಗೆರೆಗಳನ್ನು (ಕಂಬಗಳು) ಮತ್ತು **ಅಡ್ಡ ಫಿಲ್ಟರ್** (ಮಧ್ಯದ ಸಾಲು = 1) ಅಡ್ಡ ಗೆರೆಗಳನ್ನು (ಮೇಲ್ಛಾವಣಿ) ಹೈಲೈಟ್ ಮಾಡುತ್ತವೆ. ಎರಡು ಫೀಚರ್ ಮ್ಯಾಪ್‌ಗಳು ಬೇರೆ ಬೇರೆ ಮಾಹಿತಿ ತೋರಿಸುತ್ತವೆ. ಹಂಪಿಯ ಕಲ್ಲಿನ ರಥದ ಚಿತ್ರಕ್ಕೆ ಹಾಕಿದರೆ ಕಂಬಗಳು ಲಂಬ ಫಿಲ್ಟರ್‌ನಲ್ಲಿ ಮತ್ತು ಛಾವಣಿಯ ಅಂಚು ಅಡ್ಡ ಫಿಲ್ಟರ್‌ನಲ್ಲಿ ಪ್ರಕಾಶಿಸುತ್ತವೆ.

೬. Zero Padding ಮತ್ತು Stride, ಔಟ್‌ಪುಟ್ ಗಾತ್ರದ ಸೂತ್ರ

Zero Padding

ಕರ್ನಲ್ ಜಾರಿಸಿದಾಗ ಚಿತ್ರ ಚಿಕ್ಕದಾಗುತ್ತದೆ ಮತ್ತು ಅಂಚಿನ ಪಿಕ್ಸೆಲ್‌ಗಳು ಕಡಿಮೆ ಬಾರಿ ಬಳಕೆಯಾಗುತ್ತವೆ. ಇದಕ್ಕೆ ಚಿತ್ರದ ಸುತ್ತ **ಸೊನ್ನೆಗಳ ಚೌಕಟ್ಟು** ಸೇರಿಸುವುದೇ **zero padding**. Keras ನಲ್ಲಿ:

- `padding="valid"` → ಪ್ಯಾಡಿಂಗ್ ಇಲ್ಲ, ಔಟ್‌ಪುಟ್ ಚಿಕ್ಕದು.
- `padding="same"` → ಸೊನ್ನೆಗಳನ್ನು ಎಡ-ಬಲ, ಮೇಲೆ-ಕೆಳಗೆ ಸಮನಾಗಿ ಸೇರಿಸಿ, stride = 1 ಆದಾಗ ಔಟ್‌ಪುಟ್ ಗಾತ್ರ = ಇನ್‌ಪುಟ್ ಗಾತ್ರ.

> **ಹೋಲಿಕೆ: ಫೋಟೋಗೆ ಫ್ರೇಮ್**
> ಫೋಟೋದ ಅಂಚಿನ ಜನರೂ ಮಧ್ಯದವರಂತೆ ಸಮಾನವಾಗಿ ಫ್ರೇಮ್‌ನಲ್ಲಿ ಕಾಣಲಿ ಎಂದು ಸುತ್ತ ಖಾಲಿ ಅಂಚು (mat) ಹಾಕುವಂತೆ.

ಸ್ಲೈಡ್‌ನ ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆ (5 ನಿಮಿಷ): padding = "same", stride = 1

ಇನ್‌ಪುಟ್ 3×3 = [[1, 2, 3], [−1, 0, 3], [1, 2, −3]]; ಕರ್ನಲ್ ಹಿಂದಿನ K. ಸುತ್ತ ಸೊನ್ನೆ ಸೇರಿಸಿ 5×5 ಮಾಡಿ, 3×3 ಕರ್ನಲ್ ಜಾರಿಸಿದರೆ ಔಟ್‌ಪುಟ್ 3×3:

ಇನ್‌ಪುಟ್ 3×3 + ಶೂನ್ಯ ಪ್ಯಾಡಿಂಗ್ ಔಟ್‌ಪುಟ್ (3×3, same)

0	0	0	0	0
0	1	2	3	0
0	-1	0	3	0
0	1	2	-3	0
0	0	0	0	0

→

2	6	2
3	2	5
1	-2	5

ಚಿತ್ರ ೪: ಬೂದು ಅಂಚು = ಸೇರಿಸಿದ ಸೊನ್ನೆಗಳು. ಇನ್‌ಪುಟ್ ಮತ್ತು ಔಟ್‌ಪುಟ್ ಎರಡೂ 3×3

> **ಸೆಲ್ (0, 0): ಮೇಲಿನ ಎಡ ಕಿಟಕಿ (ಪ್ಯಾಡ್ ಮಾಡಿದ 5×5 ನಲ್ಲಿ)**
> ಕಿಟಕಿ = [[0, 0, 0], [0, 1, 2], [0, −1, 0]]
>
> ```
> ಸಾಲು 1: 0×0 + 0×1 + 0×0   = 0
> ಸಾಲು 2: 0×1 + 1×0 + 2×1   = 2
> ಸಾಲು 3: 0×1 + (−1)×0 + 0×1 = 0      ಮೊತ್ತ = 2
> ```
>
> ಪೂರ್ಣ ಉತ್ತರ: **[[2, 6, 2], [3, 2, 5], [1, −2, 5]]**. ಹೋಲಿಕೆ: ಇನ್‌ಪುಟ್ 3×3 → ಔಟ್‌ಪುಟ್ 3×3 (ಅದೇ ಗಾತ್ರ). “valid” ಆಗಿದ್ದರೆ 3 − 3 + 1 = 1×1 ಮಾತ್ರ ಸಿಗುತ್ತಿತ್ತು.

Stride (ಹೆಜ್ಜೆ)

Stride = ಕರ್ನಲ್ ಪ್ರತಿ ಬಾರಿ ಜಾರುವ ದೂರ. stride = 1 ಎಂದರೆ ಒಂದೊಂದು ಪಿಕ್ಸೆಲ್; stride = 2 ಎಂದರೆ ಎರಡೆರಡು — ಔಟ್‌ಪುಟ್ ಚಿಕ್ಕದಾಗುತ್ತದೆ. *ಹೋಲಿಕೆ:* ಮೆಟ್ಟಿಲು ಹತ್ತುವಾಗ ಒಂದೊಂದಾಗಿ ಅಥವಾ ಎರಡೆರಡು ಮೆಟ್ಟಿಲು ಹಾರುವುದು.

ಔಟ್‌ಪುಟ್ ಗಾತ್ರದ ಸೂತ್ರ (ಪ್ರತಿ ಆಯಾಮಕ್ಕೆ)

```
ಔಟ್‌ಪುಟ್ = ⌊ (n + 2p − f) / s ⌋ + 1

n = ಇನ್‌ಪುಟ್ ಗಾತ್ರ,  f = ಕರ್ನಲ್ ಗಾತ್ರ,  p = ಪ್ಯಾಡಿಂಗ್ (ಪ್ರತಿ ಬದಿಗೆ),  s = stride
```

(ಸ್ಲೈಡ್‌ನ ನಿಯಮ: LeNet ನಲ್ಲಿ “W − F + 1 = 32 − 5 + 1 = 28” — ಇದು p = 0, s = 1 ವಿಶೇಷ ಪ್ರಕರಣ.)

ಇನ್‌ಪುಟ್ n	ಕರ್ನಲ್ f	ಪ್ಯಾಡಿಂಗ್ p	stride s	ಲೆಕ್ಕ	ಔಟ್‌ಪುಟ್
28	3	0 (valid)	1	(28 + 0 − 3)/1 + 1	**26**
28	3	1 (same)	1	(28 + 2 − 3)/1 + 1	**28**
28	3	1	2	⌊(28 + 2 − 3)/2⌋ + 1 = ⌊13.5⌋ + 1	**14**
32	5	0	1	32 − 5 + 1 (LeNet)	**28**
6	3	0	1	6 − 3 + 1	**4**

೭. ಹಲವು ಫೀಚರ್ ಮ್ಯಾಪ್‌ಗಳು, ಚಾನೆಲ್‌ಗಳು ಮತ್ತು ಪ್ಯಾರಾಮೀಟರ್ ಉಳಿತಾಯ

- ಒಂದು convolution ಪದರದಲ್ಲಿ **ಹಲವು ಫೀಚರ್ ಮ್ಯಾಪ್‌ಗಳು** ಇರುತ್ತವೆ (3D ರಚನೆ: ಎತ್ತರ × ಅಗಲ × ಫಿಲ್ಟರ್ ಸಂಖ್ಯೆ). ಪ್ರತಿ ಮ್ಯಾಪ್ ಬೇರೆ ಫಿಲ್ಟರ್‌ನಿಂದ ಬರುತ್ತದೆ.
- ಒಂದು ಮ್ಯಾಪ್‌ನ ಎಲ್ಲ ನ್ಯೂರಾನ್‌ಗಳು **ಒಂದೇ ತೂಕ ಮತ್ತು bias** ಹಂಚಿಕೊಳ್ಳುತ್ತವೆ. ಇದರಿಂದ ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು ಕಡಿಮೆ, ಮತ್ತು ಮಾದರಿ ಚಿತ್ರದ **ಯಾವುದೇ ಸ್ಥಳದಲ್ಲಿರುವ** ಮಾದರಿಯನ್ನು ಗುರುತಿಸಬಲ್ಲದು — ಇದಕ್ಕೆ **translation invariance** ಎನ್ನುತ್ತಾರೆ.
- ಇನ್‌ಪುಟ್ ಪದರಕ್ಕೂ ಹಲವು **ಚಾನೆಲ್‌ಗಳು**: ಸಾಮಾನ್ಯವಾಗಿ RGB (Red, Green, Blue). ಉಪಗ್ರಹ ಚಿತ್ರಗಳಲ್ಲಿ infrared ನಂತಹ ಹೆಚ್ಚುವರಿ ಚಾನೆಲ್ ಇರಬಹುದು.

Conv ಪದರದ ಪ್ಯಾರಾಮೀಟರ್ ಸೂತ್ರ

```
ಪ್ಯಾರಾಮೀಟರ್ = ಫಿಲ್ಟರ್ ಸಂಖ್ಯೆ × ( ಕರ್ನಲ್ ಎತ್ತರ × ಕರ್ನಲ್ ಅಗಲ × ಇನ್‌ಪುಟ್ ಚಾನೆಲ್‌ಗಳು + 1 )
```

ಒಂದು ಫಿಲ್ಟರ್ ಇನ್‌ಪುಟ್‌ನ **ಎಲ್ಲ ಚಾನೆಲ್‌ಗಳನ್ನೂ** ನೋಡುತ್ತದೆ (ಆದ್ದರಿಂದ 3×3×ಚಾನೆಲ್ ಆಕಾರದ ಘನ ಫಿಲ್ಟರ್), ಮತ್ತು ಪ್ರತಿ ಫಿಲ್ಟರ್‌ಗೆ ಒಂದು bias (+1).

ಲೆಕ್ಕ: ಬಣ್ಣದ ಚಿತ್ರದ ಮೊದಲ ಪದರ

RGB ಇನ್‌ಪುಟ್ (3 ಚಾನೆಲ್), 3×3 ಫಿಲ್ಟರ್, 32 ಫಿಲ್ಟರ್‌ಗಳು:

```
ಒಂದು ಫಿಲ್ಟರ್: 3×3×3 = 27 ತೂಕ + 1 bias = 28
32 ಫಿಲ್ಟರ್: 32 × 28 = 896 ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು
```

ಈ 896 ಸಂಖ್ಯೆ ಚಿತ್ರ 32×32 ಆಗಿರಲಿ, 4000×4000 ಆಗಿರಲಿ ಒಂದೇ — ಏಕೆಂದರೆ ಫಿಲ್ಟರ್‌ಗಳು ಚಿತ್ರದ ಗಾತ್ರದ ಮೇಲೆ ಅವಲಂಬಿತವಲ್ಲ.

೮. Pooling ಪದರ

Pooling (subsampling) ಫೀಚರ್ ಮ್ಯಾಪ್‌ನ ಎತ್ತರ-ಅಗಲವನ್ನು ಕುಗ್ಗಿಸಿ ಲೆಕ್ಕಾಚಾರ ಕಡಿಮೆ ಮಾಡುತ್ತದೆ, overfitting ತಗ್ಗಿಸುತ್ತದೆ, ಮತ್ತು ವಸ್ತುವಿನ ಗಾತ್ರ/ಸ್ಥಾನ ಸ್ವಲ್ಪ ಬದಲಾದರೂ ಗುರುತಿಸಲು ನೆರವಾಗುತ್ತದೆ. ಪ್ರತಿ ನ್ಯೂರಾನ್ ಹಿಂದಿನ ಪದರದ ಒಂದು ಚಿಕ್ಕ ಭಾಗದಿಂದ (ಉದಾ: 2×2) ಮೌಲ್ಯಗಳನ್ನು max ಅಥವಾ average ಮೂಲಕ ಒಂದುಗೂಡಿಸುತ್ತದೆ.

- **Max pooling:** ಪ್ರತಿ ಭಾಗದ **ಅತಿ ದೊಡ್ಡ** ಮೌಲ್ಯ ಮಾತ್ರ ಮುಂದಕ್ಕೆ; ಉಳಿದವು ತಿರಸ್ಕೃತ.
- 2×2 ಫಿಲ್ಟರ್, stride = 2 → ಔಟ್‌ಪುಟ್ ವಿಸ್ತೀರ್ಣ 4 ಪಟ್ಟು ಚಿಕ್ಕ; ಕೇವಲ **25%** ಮೌಲ್ಯಗಳು ಉಳಿಯುತ್ತವೆ.
- Pooling ಪ್ರತಿ ಚಾನೆಲ್‌ಗೆ ಪ್ರತ್ಯೇಕವಾಗಿ ಅನ್ವಯಿಸುತ್ತದೆ → ಆಳ (ಚಾನೆಲ್ ಸಂಖ್ಯೆ) ಬದಲಾಗುವುದಿಲ್ಲ. **ಇದಕ್ಕೆ ಕಲಿಯಬಲ್ಲ ಪ್ಯಾರಾಮೀಟರ್‌ಗಳಿಲ್ಲ (0).**

ಹೋಲಿಕೆ: ಟಾಪರ್ ಮಾತ್ರ ಮುಂದಿನ ಹಂತಕ್ಕೆ

ಪ್ರತಿ ಬೆಂಚ್‌ನ (2×2 = 4 ವಿದ್ಯಾರ್ಥಿಗಳು) ಅತಿ ಹೆಚ್ಚು ಅಂಕ ಪಡೆದವನನ್ನು ಮಾತ್ರ ತಾಲೂಕು ಮಟ್ಟಕ್ಕೆ ಕಳುಹಿಸುವಂತೆ (max pooling). ಅಥವಾ ಗೂಗಲ್ ಮ್ಯಾಪ್ ಅನ್ನು zoom-out ಮಾಡಿದಾಗ ಸಣ್ಣ ವಿವರಗಳು ಹೋಗಿ ದೊಡ್ಡ ಚಿತ್ರ ಮಾತ್ರ ಉಳಿಯುವಂತೆ.

ಸ್ಲೈಡ್‌ನ ಅಭ್ಯಾಸ (5 ನಿಮಿಷ): 2×2 max pooling, stride = 2, ಅದೇ 6×6 ಮ್ಯಾಟ್ರಿಕ್ಸ್

ಇನ್‌ಪುಟ್ (6×6)

1	2	3	1	2	3
-1	0	3	0	1	2
1	2	-3	1	-1	0
-1	2	3	0	-1	3
1	0	-3	1	3	-2
-2	1	0	1	2	1

→ max –

Max Pooling ಔಟ್‌ಪುಟ್ (3×3)

2	3	3
2	3	3
1	1	3

ಚಿತ್ರ ೫: ಪ್ರತಿ ಬಣ್ಣದ 2×2 ಬ್ಲಾಕ್‌ನ ಗರಿಷ್ಠ ಮಾತ್ರ ಔಟ್‌ಪುಟ್‌ಗೆ

ಹಂತ-ಹಂತ

```
ಬ್ಲಾಕ್ (0,0): [[ 1, 2], [-1, 0]] → max = 2
ಬ್ಲಾಕ್ (0,1): [[ 3, 1], [ 3, 0]] → max = 3
ಬ್ಲಾಕ್ (0,2): [[ 2, 3], [ 1, 2]] → max = 3
ಬ್ಲಾಕ್ (1,0): [[ 1, 2], [-1, 2]] → max = 2
ಬ್ಲಾಕ್ (1,1): [[-3, 1], [ 3, 0]] → max = 3
ಬ್ಲಾಕ್ (1,2): [[-1, 0], [-1, 3]] → max = 3
ಬ್ಲಾಕ್ (2,0): [[ 1, 0], [-2, 1]] → max = 1
ಬ್ಲಾಕ್ (2,1): [[-3, 1], [ 0, 1]] → max = 1
ಬ್ಲಾಕ್ (2,2): [[ 3,-2], [ 2, 1]] → max = 3
```

ಉತ್ತರ (3×3): [[2, 3, 3], [2, 3, 3], [1, 1, 3]]. 36 ಮೌಲ್ಯಗಳಿಂದ 9 (= 25%).

ಸ್ಥಾನ-ಸ್ವಾತಂತ್ರ್ಯದ ಉದಾಹರಣೆ: ಒಂದು ಬ್ಲಾಕ್‌ನಲ್ಲಿ ಪ್ರಕಾಶಮಾನ ಪಿಕ್ಸೆಲ್ [[9, 0], [0, 0]] ಇದ್ದರೂ, ಅದು [[0, 0], [0, 9]] ಗೆ ಸ್ವಲ್ಪ ಸರಿದರೂ max = 9 ಅದೇ — ಸ್ವಲ್ಪ ಸರಿಯುವಿಕೆಗೆ ನೆಟ್‌ವರ್ಕ್ ಅಸಂವೇದಿ.

Average pooling ಬ್ಲಾಕ್‌ನ ಸರಾಸರಿ ತೆಗೆದುಕೊಳ್ಳುತ್ತದೆ (ಉದಾ: ಮೊದಲ ಬ್ಲಾಕ್ (1+2−1+0)/4 = 0.5). LeNet-5 ಇದನ್ನು ಬಳಸುತ್ತದೆ; ಆಧುನಿಕ CNN ಗಳು ಹೆಚ್ಚಾಗಿ max pooling. Keras ನಲ್ಲಿ `MaxPooling1D/2D`, `AveragePooling1D/2D` (ಮುಖ್ಯ ಆರ್ಗ್ಯುಮೆಂಟ್: `pool_size`, `strides`, `padding`).

೯. LeNet-5 ಮತ್ತು Conv ಪದರದ ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು

LeNet-5 (Yann LeCun, Léon Bottou, Patrick Haffner, Yoshua Bengio) — ಮೊದಲ ಪ್ರಸಿದ್ಧ CNN, ಕೈಬರಹದ ಅಂಕೆ ಗುರುತಿಸಲು: 32×32 ಇನ್‌ಪುಟ್ → Conv → Pool → Conv → Pool → Dense (120, 84) → 10 ಔಟ್‌ಪುಟ್.

```
model = models.Sequential()
model.add(layers.Conv2D(6, (5, 5), activation='relu', input_shape=(32, 32, 1),
padding='valid'))
# (32 − 5 + 1) × (32 − 5 + 1) = 28 × 28, 6 ಫೀಚರ್ ಮ್ಯಾಪ್
model.add(layers.AveragePooling2D(pool_size=(2, 2)))      # 28×28 → 14×14
model.add(layers.Conv2D(16, (5, 5), activation='relu', padding='valid'))   # 14 → 10,
16 ಮ್ಯಾಪ್
model.add(layers.AveragePooling2D(pool_size=(2, 2)))      # 10×10 → 5×5
model.add(layers.Flatten())                               # 5×5×16 = 400
model.add(layers.Dense(120, activation='relu'))
model.add(layers.Dense(84, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
```

ಪದರ	ಔಟ್‌ಪುಟ್ ಆಕಾರ	ಪ್ಯಾರಾಮೀಟರ್ ಲೆಕ್ಕ	ಪ್ಯಾರಾ.
Input	32×32×1	—	0
Conv2D(6, 5×5)	28×28×6	6 × (5×5×1 + 1) = 6 × 26	156
AvgPool 2×2	14×14×6	—	0
Conv2D(16, 5×5)	10×10×16	16 × (5×5×6 + 1) = 16 × 151	2,416
AvgPool 2×2	5×5×16	—	0
Flatten	400	—	0
Dense(120)	120	400 × 120 + 120	48,120
Dense(84)	84	120 × 84 + 84	10,164
Dense(10)	10	84 × 10 + 10	850
ಒಟ್ಟು			**61,706**

ಗಮನಿಸಿ: ನಿಜವಾದ ಮೂಲ LeNet-5 ನಲ್ಲಿ ಕೆಲವು ಸಂಪರ್ಕಗಳು ವಿರಳ; ಇಲ್ಲಿ ಸ್ಲೈಡ್‌ನ Keras ರೂಪ (ಸಂಪೂರ್ಣ ಸಂಪರ್ಕ) ಬಳಸಿ ಲೆಕ್ಕ ಹಾಕಿದ್ದೇವೆ.

Conv2D ಪದರದ ಮುಖ್ಯ ಆರ್ಗ್ಯುಮೆಂಟ್‌ಗಳು

ಉದಾಹರಣೆ ಕರೆ: `Conv2D(filters=32, kernel_size=(3,3), strides=(1,1), padding="same", activation="relu")`

ಆರ್ಗ್ಯುಮೆಂಟ್	ಅರ್ಥ	ಉದಾಹರಣೆ ಮೌಲ್ಯ
`filters`	ಫಿಲ್ಟರ್ ಸಂಖ್ಯೆ = ಔಟ್‌ಪುಟ್ ಚಾನೆಲ್‌ಗಳು = ಕಲಿಯುವ ಫೀಚರ್ ಮ್ಯಾಪ್‌ಗಳು	32 ಫೀಚರ್ ಮ್ಯಾಪ್‌ಗಳು
`kernel_size`	ಫಿಲ್ಟರ್‌ನ ಗಾತ್ರ	3×3 ಪ್ರದೇಶ ನೋಡುತ್ತದೆ
`strides`	ಪ್ರತಿ ಬಾರಿ ಜಾರುವ ಹೆಜ್ಜೆ	(1,1): ಒಂದೊಂದು ಪಿಕ್ಸೆಲ್
`padding`	ಸೊನ್ನೆ ಸೇರಿಸಬೇಕೇ? "valid" / "same"	"same": ಗಾತ್ರ ಉಳಿಯುತ್ತದೆ
`activation`	ಸಕ್ರಿಯಕರಣ ಫಂಕ್ಷನ್ (non-linearity)	ReLU
`use_bias`	bias ಸೇರಿಸಬೇಕೇ? ಸಾಮಾನ್ಯವಾಗಿ True	—
`kernel_initializer`, `bias_initializer`	ತೂಕ/bias ಆರಂಭಿಕ ಮೌಲ್ಯ	ಡೀಫಾಲ್ಟ್: kernel = "glorot_uniform", bias = ಸೊನ್ನೆಗಳು

Conv1D ಕೂಡ ಇದೇ ರೀತಿಯ ಆರ್ಗ್ಯುಮೆಂಟ್‌ಗಳನ್ನು ಬಳಸುತ್ತದೆ — ಅಲ್ಲಿ kernel_size ಒಂದೇ ಸಂಖ್ಯೆ (ಉದಾ: 5), ಅನುಕ್ರಮ (ಪಠ್ಯ, ಸಂಕೇತ) ಮೇಲೆ ಜಾರುತ್ತದೆ.

೧೦. MNIST ಅಂಕೆ ಗುರುತಿಸುವಿಕೆ — CNN ಕೋಡ್ ಮತ್ತು ಪ್ಯಾರಾಮೀಟರ್ ಲೆಕ್ಕ

ಹಿಂದಿನ ಉಪನ್ಯಾಸದಲ್ಲಿ MNIST ಅನ್ನು Dense ನೆಟ್‌ವರ್ಕ್‌ನಿಂದ (79,510 ಪ್ಯಾರಾಮೀಟರ್) ಗುರುತಿಸಿದ್ದೆವು. ಈಗ CNN ಬಳಸೋಣ — ಚಿತ್ರದ ಆಕಾರ (28×28) ಉಳಿಸಿಕೊಂಡೇ ಕಲಿಯುತ್ತದೆ.

```
import tensorflow as tf, numpy as np, matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras.layers import Input, Dense, Flatten, Conv2D, MaxPooling2D

mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()   # 60,000 ತರಬೇತಿ, 10,000 ಪರೀಕ್ಷೆ

x_train = x_train / 255            # ಪಿಕ್ಸೆಲ್ ಮೌಲ್ಯಗಳನ್ನು 0–1 ವ್ಯಾಪ್ತಿಗೆ (255 ರಿಂದ ಭಾಗಿಸಿ)
x_test  = x_test  / 255
y_train2 = keras.utils.to_categorical(y_train, 10)   # ಲೇಬಲ್ → one-hot: 2 →
[0,0,1,0,0,0,0,0,0,0]
y_test2  = keras.utils.to_categorical(y_test, 10)

model = keras.Sequential([
    Input(shape=(28, 28, 1)),                                    # (batch, 28, 28, 1)
    Conv2D(32, (3, 3), padding='same', activation='relu'),     # → (batch, 28, 28,
32)
    MaxPooling2D((2, 2), strides=2),                             # → (batch, 14, 14,
32)
    Conv2D(64, (3, 3), padding='same', activation='relu'),     # → (batch, 14, 14,
64)
    MaxPooling2D((2, 2), strides=2),                             # → (batch, 7, 7, 64)
    Flatten(),                                                   # → (batch, 3136)
    Dense(100, activation='relu'),
    Dense(10, activation='softmax')
])
print(model.summary())             # ಮಾದರಿ ಸಾರಾಂಶ

opt = keras.optimizers.Adam(learning_rate=0.01)
model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
mokymo = model.fit(x_train, y_train2, batch_size=32, epochs=6, validation_split=0.2)
```

(ಸ್ಲೈಡ್‌ನಲ್ಲಿ ಅಂತಿಮ ಸಾಲಿನ ವೇರಿಯಬಲ್ ಹೆಸರು “mokymo” ಎಂದಿದೆ (ಲಿಥುವೇನಿಯನ್ “ತರಬೇತಿ”) — ನಾವು ಅದನ್ನೇ ಇಟ್ಟಿದ್ದೇವೆ; ನೀವು `history` ಎಂದೂ ಹೆಸರಿಡಬಹುದು. ಸ್ಲೈಡ್‌ನಲ್ಲಿ learning rate 0.01; ಹಿಂದಿನ ಉಪನ್ಯಾಸದಲ್ಲಿ 0.001 ಬಳಸಿದ್ದೆವು.)

ಪ್ಯಾರಾಮೀಟರ್ ಲೆಕ್ಕ (ಸ್ಲೈಡ್‌ನ model.summary())

ಪದರ	ಔಟ್‌ಪುಟ್ ಆಕಾರ	ಲೆಕ್ಕ	ಪ್ಯಾರಾ.
Conv2D(32, 3×3)	(28, 28, 32)	32 × (3×3×1 + 1) = 32 × 10	320
MaxPooling2D	(14, 14, 32)	—	0
Conv2D(64, 3×3)	(14, 14, 64)	64 × (3×3×32 + 1) = 64 × 289 (ಇನ್‌ಪುಟ್ 32 ಚಾನೆಲ್)	18,496
MaxPooling2D	(7, 7, 64)	—	0
Flatten	(3136)	7 × 7 × 64 = 3136	0
Dense(100)	(100)	(3136 + 1) × 100	313,700
Dense(10)	(10)	(100 + 1) × 10	1,010
ಒಟ್ಟು			**333,526** (≈ 1.27 MB, ಎಲ್ಲವೂ trainable)

> **ಆಸಕ್ತಿದಾಯಕ ಒಳನೋಟ (ನನ್ನ ವಿಶ್ಲೇಷಣೆ)**
> Dense(100) ಪದರದ 313,700 ಪ್ಯಾರಾಮೀಟರ್ ಒಟ್ಟು 333,526 ರ **ಸುಮಾರು 94%**. ಅಂದರೆ ಚಿತ್ರದ ಲಕ್ಷಣ ಹೊರತೆಗೆಯುವ ಎರಡು Conv ಪದರಗಳು ಕೇವಲ 18,816 (≈ 6%) ಪ್ಯಾರಾಮೀಟರ್ ಬಳಸಿವೆ. ಇದೇ CNN ನ ಶಕ್ತಿ: ಕಡಿಮೆ ಪ್ಯಾರಾಮೀಟರ್‌ನಿಂದ ಚಿತ್ರದ ಸ್ಥಳೀಯ ಮಾದರಿ ಕಲಿಯುವುದು. ಅಲ್ಲದೆ pooling ನಿಂದ ಫ್ಲ್ಯಾಟನ್‌ಗೆ ಮೊದಲು ಗಾತ್ರ 28×28 ರಿಂದ 7×7 ಕ್ಕೆ ಕುಗ್ಗದಿದ್ದರೆ Flatten = 28×28×64 = 50,176 ಆಗಿ Dense ಪ್ಯಾರಾಮೀಟರ್ ≈ 5 ಮಿಲಿಯನ್ ಆಗುತ್ತಿತ್ತು.

೧೧. ಪಠ್ಯಕ್ಕೆ CNN: IMDB ವಿಮರ್ಶೆಗಳು (Conv1D)

CNN ಕೇವಲ ಚಿತ್ರಕ್ಕಲ್ಲ — **ಅನುಕ್ರಮ** (ಪಠ್ಯ, ಧ್ವನಿ, ಸಮಯ-ಸರಣಿ) ಕ್ಕೂ ಬಳಸಬಹುದು, ಆಗ **Conv1D** ಬಳಸುತ್ತೇವೆ. ಹಿಂದಿನ ಉಪನ್ಯಾಸದ IMDB ಸಮಸ್ಯೆಯನ್ನೇ (ವಿಮರ್ಶೆ positive ಯೋ negative ಯೋ) ಈಗ CNN ನಿಂದ.

> **ಹೋಲಿಕೆ: ಪದಗುಚ್ಛ ಹುಡುಕುವ ಕಿಟಕಿ**
> ವಿಮರ್ಶೆಯ ಮೇಲೆ 5 ಪದಗಳ ಕಿಟಕಿ ಜಾರಿಸುತ್ತಾ "ತುಂಬ ಚೆನ್ನಾಗಿದೆ", "ಹಣ ವ್ಯರ್ಥ", "never watch this" ನಂತಹ **ಪದಗುಚ್ಛಗಳನ್ನು** ಹುಡುಕುವ ಫಿಲ್ಟರ್‌ಗಳು. ಒಂದು ಫಿಲ್ಟರ್ ಒಂದು ಬಗೆಯ ಪದಗುಚ್ಛ-ಪತ್ತೆಕಾರ (n-gram detector). Pooling ಆ ಪದಗುಚ್ಛ ವಿಮರ್ಶೆಯ ಎಲ್ಲೇ ಇದ್ದರೂ ಹಿಡಿಯುತ್ತದೆ.

```
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Embedding, Conv1D, MaxPooling1D,
Flatten
from tensorflow.keras.preprocessing.sequence import pad_sequences

max_words = 10000
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_words)   # 25,000
+ 25,000 ವಿಮರ್ಶೆ
maxlen = 200                                    # ಎಲ್ಲ ವಿಮರ್ಶೆಗಳು 200 ಪದಗಳಿಗೆ (ಚಿಕ್ಕದಾದರೆ ಮುಂದೆ
0 ಸೇರ್ಪಡೆ)
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test  = pad_sequences(x_test,  maxlen=maxlen)

model = Sequential()
model.add(Input(shape=(maxlen,)))
model.add(Embedding(max_words, 8))              # → (batch, 200, 8): ಪ್ರತಿ ಪದಕ್ಕೆ 8 ಉದ್ದದ
ಸದಿಶ
model.add(Conv1D(100, 5, activation='relu'))    # 100 ಫಿಲ್ಟರ್, ಕಿಟಕಿ 5 → (batch, 200 − 5
+ 1 = 196, 100)
model.add(MaxPooling1D(pool_size=2))            # → (batch, 98, 100)
model.add(Conv1D(200, 5, activation='relu'))    # → (batch, 94, 200)
model.add(MaxPooling1D(pool_size=2))            # → (batch, 47, 200)
model.add(Flatten())                            # → (batch, 9400)
model.add(Dense(128, activation='relu'))        # fully connected ಪದರ
model.add(Dense(1, activation='sigmoid'))       # 0–1 ನಡುವಿನ ಔಟ್‌ಪುಟ್: positive ಸಾಧ್ಯತೆ

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.summary()

history = model.fit(x_train, y_train, epochs=5, batch_size=128, validation_split=0.1)
scores  = model.evaluate(x_test, y_test, verbose=1)
```

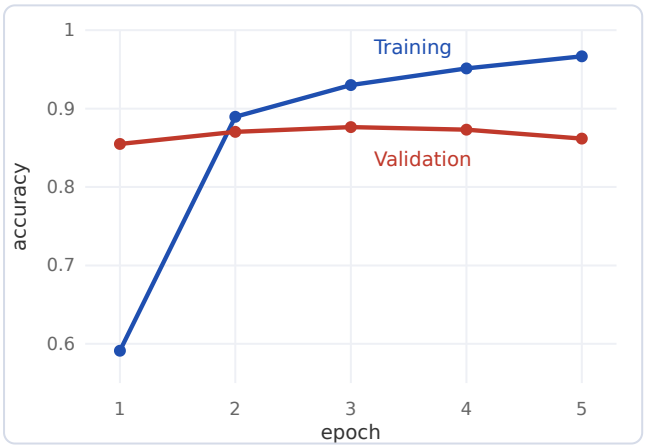
ಪದರ	ಔಟ್‌ಪುಟ್ ಆಕಾರ	ಲೆಕ್ಕ	ಪ್ಯಾರಾ.
Embedding(10000, 8)	(200, 8)	10,000 × 8	80,000
Conv1D(100, 5)	(196, 100)	100 × (5×8 + 1) = 100 × 41	4,100
MaxPooling1D(2)	(98, 100)	—	0
Conv1D(200, 5)	(94, 200)	200 × (5×100 + 1) = 200 × 501	100,200
MaxPooling1D(2)	(47, 200)	—	0
Flatten	(9400)	47 × 200	0
Dense(128)	(128)	9400 × 128 + 128	1,203,328
Dense(1)	(1)	128 + 1	129
ಒಟ್ಟು			**1,387,757**

Conv1D ನ ಇನ್‌ಪುಟ್ ಚಾನೆಲ್ = embedding ಆಯಾಮ (8): ಪ್ರತಿ ಫಿಲ್ಟರ್ 5 ಪದಗಳ × 8 ಸಂಖ್ಯೆಗಳನ್ನು ನೋಡುತ್ತದೆ (40 ತೂಕ + 1 bias).

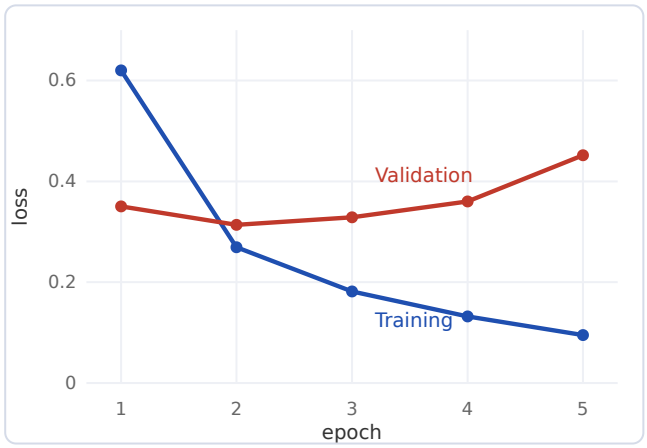
ತರಬೇತಿ ಫಲಿತಾಂಶ (ಸ್ಲೈಡ್ 32–33 ರ ನಿಜವಾದ ಸಂಖ್ಯೆಗಳು)

Epoch	loss	accuracy	val_loss	val_accuracy
1	0.6201	0.5912	0.3505	0.8548
2	0.2692	0.8894	**0.3138** (ಕನಿಷ್ಠ)	0.8704
3	0.1818	0.9300	0.3286	**0.8764** (ಗರಿಷ್ಠ)
4	0.1322	0.9512	0.3603	0.8732
5	0.0951	0.9667	0.4517	0.8616

Test: loss = 0.4896, accuracy = 0.8502 (85.0%). ಹೆಜ್ಜೆಗಳು: 25,000 × 0.9 = 22,500 ತರಬೇತಿ; 22,500 / 128 = 175.8 → **176 ಹೆಜ್ಜೆ/epoch** (ಲಾಗ್‌ನಲ್ಲಿ 176/176 ✓). Test: 25,000 / 32 = 781.25 → **782** (ಲಾಗ್‌ನಲ್ಲಿ 782/782 ✓).



ಚಿತ್ರ ೬: Accuracy — training ಏರುತ್ತಲೇ, validation epoch 3 ರ ನಂತರ ಇಳಿಯುತ್ತದೆ



ಚಿತ್ರ ೭: Loss — validation loss epoch 2 ರ ನಂತರ ಏರುತ್ತದೆ

ಇಲ್ಲಿ ಸ್ಪಷ್ಟ overfitting ಇದೆ!
Training accuracy 59% → 96.7% ಏರಿದರೂ validation accuracy 87.6% (epoch 3) ರಿಂದ 86.2% ಕ್ಕೆ ಇಳಿಯಿತು; validation loss 0.3138 ರಿಂದ 0.4517 ಕ್ಕೆ ಏರಿತು (training loss 0.0951 ಕ್ಕೆ ಇಳಿದಾಗ!). ಅಂತರ: 96.7% vs 86.2% ≈ 10 ಅಂಶ. 1.39 ಮಿಲಿಯನ್ ಪ್ಯಾರಾಮೀಟರ್ ಮಾದರಿ 22,500 ವಿಮರ್ಶೆಗಳನ್ನು ಉರು ಹೊಡೆಯುತ್ತಿದೆ. **ಪರಿಹಾರ (ಹಿಂದಿನ ಉಪನ್ಯಾಸದಿಂದ):** epoch 3 ರಲ್ಲಿ early stopping, Dropout, L2 regularisation, ಚಿಕ್ಕ Dense ಪದರ. ಒಟ್ಟು ಪ್ಯಾರಾಮೀಟರ್‌ನಲ್ಲಿ Dense(128) ಪದರವೇ 1,203,328 (≈ 87%).

೧೨. ಪ್ರಸಿದ್ಧ ವಿನ್ಯಾಸಗಳು: AlexNet, VGG16, ResNet

AlexNet (2012)

ಇನ್‌ಪುಟ್ 224×224×3; ದೊಡ್ಡ ಕರ್ನಲ್ (11×11, stride 4) ನಿಂದ ಆರಂಭ, ReLU, max pooling, ನಂತರದ ಪದರಗಳಲ್ಲಿ 5×5 ಮತ್ತು 3×3 ಕರ್ನಲ್; ಕೊನೆಯಲ್ಲಿ ಎರಡು 4096 Dense ಪದರ (Dropout 0.5 ಸಹಿತ) ಮತ್ತು 1000 ವರ್ಗಗಳ softmax. ReLU ಮತ್ತು Dropout ನಂತಹ ತಂತ್ರಗಳನ್ನು ದೊಡ್ಡ ಪ್ರಮಾಣದಲ್ಲಿ ಬಳಸಿ ಯಶಸ್ವಿಯಾಯಿತು (ಸ್ಲೈಡ್ 2 ರ ಆರ್ಕಿಟೆಕ್ಚರ್ ಕೋಷ್ಟಕ).

VGG16 (2014)

ಕರೆನ್ ಸಿಮೊನ್ಯಾನ್ ಮತ್ತು ಆಂಡ್ರ್ಯೂ ಝಿಸ್ಸರ್ಮನ್ (Visual Geometry Group, Oxford). ILSVRC-2014 ರಲ್ಲಿ ImageNet ನಲ್ಲಿ **92.7%** (top-5) ನಿಖರತೆ. “16” = ತೂಕ ಇರುವ ಪದರಗಳ ಸಂಖ್ಯೆ: **13 convolution + 3 fully connected**. ಎಲ್ಲ conv ಪದರಗಳು ಕೇವಲ **3×3** ಫಿಲ್ಟರ್ (stride 1, same padding), ಕೆಲವು conv ನಂತರ 2×2 max pooling; ಆಳ ಹೆಚ್ಚಿದಂತೆ ಫಿಲ್ಟರ್ ಸಂಖ್ಯೆ 64 → 128 → 256 → 512; ಕೊನೆಯಲ್ಲಿ Dense 4096, 4096, 1000 (softmax). ಒಟ್ಟು ಸುಮಾರು **138 ಮಿಲಿಯನ್** ಪ್ಯಾರಾಮೀಟರ್ (VGG ಪೇಪರ್‌ನ Table 2, ಕಾನ್ಫಿಗರೇಶನ್ D).

> **VGG ಯ ಜಾಣ ಕಲ್ಪನೆ: ಒಂದು 7×7 ಬದಲು ಮೂರು 3×3 (ಸ್ಲೈಡ್ 36 ರ ಪೇಪರ್ ಭಾಗ)**
> ಮೂರು 3×3 ಪದರಗಳನ್ನು ಜೋಡಿಸಿದರೆ ಪರಿಣಾಮಕಾರಿ receptive field ಒಂದು 7×7 ಪದರದಷ್ಟೇ (3×3 → 3, ಎರಡು → 5×5, ಮೂರು → 7×7). ಇನ್‌ಪುಟ್-ಔಟ್‌ಪುಟ್ ಎರಡೂ C ಚಾನೆಲ್ ಆದರೆ (bias ಬಿಟ್ಟು):
>
> ```
> ಮೂರು 3×3 ಪದರ : 3 × (3×3×C×C) = 27 C²
> ಒಂದು 7×7 ಪದರ : 7×7×C×C          = 49 C²
> C = 64 ಆದರೆ    : 27 × 4096 = 110,592   vs   49 × 4096 = 200,704    → ಸುಮಾರು 45% ಕಡಿಮೆ
> ```
>
> ಲಾಭಗಳು: ಕಡಿಮೆ ಪ್ಯಾರಾಮೀಟರ್ (ಒಂದು ರೀತಿಯ regularisation) ಮತ್ತು ಮೂರು non-linearity (ReLU) ಇರುವುದರಿಂದ ನಿರ್ಧಾರ ಫಂಕ್ಷನ್ ಹೆಚ್ಚು ವಿವೇಚನಾತ್ಮಕ.

Keras ನಲ್ಲಿ ಮೊದಲೇ ತರಬೇತಿಯಾದ VGG16 ಬಳಸುವುದು

Keras ನಲ್ಲಿ ImageNet ದತ್ತಾಂಶದಲ್ಲಿ (ಸುಮಾರು 12 ಲಕ್ಷ ತರಬೇತಿ ಚಿತ್ರಗಳು, 1000 ವರ್ಗಗಳು) ಈಗಾಗಲೇ ತರಬೇತಿಯಾದ VGG16 ಲಭ್ಯ — ನಾವು ಸ್ವತಃ ತರಬೇತಿ ಮಾಡಬೇಕಿಲ್ಲ.

```
import numpy as np
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from PIL import Image

model = VGG16(weights="imagenet", include_top=True)   # include_top=True: ಕೊನೆಯ 3
Dense ಪದರ ಸೇರಿಸಿ; 1000 ವರ್ಗ
# ಇತರ ಆರ್ಗ್ಯುಮೆಂಟ್: input_shape, pooling (None/'avg'/'max'), classes=1000,
classifier_activation='softmax'

img = Image.open("dog.jpg").resize((224, 224))        # VGG16 ಗೆ ಚಿತ್ರ 224×224 ಆಗಿರಬೇಕು
x = np.array(img)                                     # (224, 224, 3)
x = np.expand_dims(x, axis=0)                         # (1, 224, 224, 3): ಒಂದು ಚಿತ್ರದ
batch
x = preprocess_input(x)                               # VGG ಗೆ ಬೇಕಾದ ಪೂರ್ವ-ಸಂಸ್ಕರಣೆ

res = model.predict(x)                                # 1000 ಸಂಭವನೀಯತೆಗಳು
print(np.argmax(res))                                 # ಸ್ಲೈಡ್‌ನ ಫಲಿತಾಂಶ: 161
```

ಸ್ಲೈಡ್‌ನ ಉದಾಹರಣೆಯಲ್ಲಿ ಒಂದು ನಾಯಿಯ ಚಿತ್ರಕ್ಕೆ ಔಟ್‌ಪುಟ್ **161**; ImageNet ವರ್ಗ ಪಟ್ಟಿಯಲ್ಲಿ 161 = “basset, basset hound” (ಒಂದು ನಾಯಿ ತಳಿ) ✓. 1000 ವರ್ಗಗಳಲ್ಲಿ ಹಲವು ನಾಯಿ ತಳಿಗಳಿವೆ (papillon, toy terrier, Rhodesian ridgeback, Afghan hound, basset hound, beagle…).

ಸರಳೀಕೃತ VGG16 (MNIST ಗೆ, ಸ್ಲೈಡ್ 40)

```
model = Sequential()
model.add(Input((28, 28, 1)))
model.add(ZeroPadding2D((2, 2)))                                          # 28×28 →
32×32
model.add(Conv2D(64,  (3,3), strides=1, padding='same', activation='relu'))
model.add(Conv2D(64,  (3,3), strides=1, padding='same', activation='relu'))
model.add(MaxPool2D((2,2), strides=2))                                    # → 16×16
model.add(Conv2D(128, (3,3), strides=1, padding='same', activation='relu'))
model.add(Conv2D(128, (3,3), strides=1, padding='same', activation='relu'))
model.add(MaxPool2D((2,2), strides=2))                                    # → 8×8
model.add(Conv2D(256, (3,3), strides=1, padding='same', activation='relu'))
model.add(Conv2D(256, (3,3), strides=1, padding='same', activation='relu'))
model.add(Conv2D(256, (3,3), strides=1, padding='same', activation='relu'))
model.add(MaxPool2D((2,2), strides=2))                                    # → 4×4×256
model.add(Flatten())                                                      # 4×4×256 =
4096
model.add(Dense(4096, activation='relu'))
model.add(Dense(128,  activation='relu'))
model.add(Dense(10,   activation='softmax'))

opt = SGD(lr=1e-4, momentum=0.9)       # ಸ್ಲೈಡ್‌ನ ಕೋಡ್ (ಹೊಸ Keras ನಲ್ಲಿ lr ಬದಲು
learning_rate)
model.compile(loss='categorical_crossentropy', optimizer=opt, metrics=['accuracy'])
model.fit(X_train, Y_train_req, epochs=20, batch_size=32, validation_split=0.1)
```

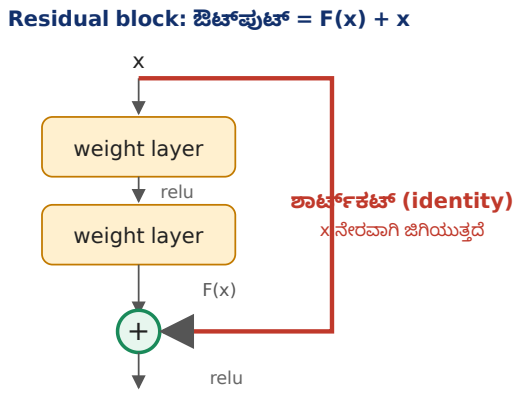
ಪ್ಯಾರಾಮೀಟರ್ ಲೆಕ್ಕ (ನನ್ನ ಲೆಕ್ಕ)

```
Conv 64 (1→64)     : 64 × (9×1   + 1) =       640
Conv 64 (64→64)    : 64 × (9×64  + 1) =    36,928
Conv 128 (64→128)  : 128× (9×64  + 1) =    73,856
Conv 128 (128→128): 128× (9×128 + 1) =   147,584
Conv 256 (128→256): 256× (9×128 + 1) =   295,168
Conv 256 ×2        : 2 × 590,080      = 1,180,160
Dense 4096         : 4096×4096 + 4096 = 16,781,312
Dense 128          : 4096×128  + 128  =    524,416
Dense 10           : 128×10    + 10   =      1,290
ಒಟ್ಟು                                ≈ 19,041,354  (ಸುಮಾರು 1.9 ಕೋಟಿ)
```

ಇಲ್ಲಿ Conv ಪದರಗಳು ಕೇವಲ 1,734,336 (≈ 9%); Dense ಪದರಗಳು ≈ 91% — ಮತ್ತೆ Dense ಭಾಗವೇ ಪ್ಯಾರಾಮೀಟರ್ ಭಾರ.

ResNet — ಅತಿ ಆಳದ ನೆಟ್‌ವರ್ಕ್‌ಗಳ ಕ್ರಾಂತಿ (2015)

ಸಮಸ್ಯೆ: ಪದರಗಳನ್ನು ಹೆಚ್ಚಿಸಿದರೆ ಕೆಲವೊಮ್ಮೆ ತರಬೇತಿ ದೋಷವೇ ಹೆಚ್ಚಾಗುತ್ತದೆ (degradation) ಮತ್ತು vanishing gradient ಕಾಡುತ್ತದೆ. **ಪರಿಹಾರ (He, Zhang, Ren, Sun — Microsoft Research):** ಪದರಗಳು ನೇರವಾಗಿ ಬೇಕಾದ ಮ್ಯಾಪಿಂಗ್ H(x) ಅನ್ನು ಕಲಿಯುವ ಬದಲು **ಉಳಿಕೆ (residual)** F(x) = H(x) − x ಅನ್ನು ಕಲಿಯಲಿ; ಔಟ್‌ಪುಟ್ = **F(x) + x**. ಇನ್‌ಪುಟ್ x ಅನ್ನು "ಶಾರ್ಟ್‌ಕಟ್ ಸಂಪರ್ಕ"ದ ಮೂಲಕ ನೇರವಾಗಿ ಕೊನೆಗೆ ಸೇರಿಸಲಾಗುತ್ತದೆ. ಅತ್ಯುತ್ತಮ ಮ್ಯಾಪಿಂಗ್ identity ಆಗಿದ್ದರೆ, ಹಲವು ಪದರಗಳನ್ನು identity ಮಾಡುವುದಕ್ಕಿಂತ residual ಅನ್ನು 0 ಕ್ಕೆ ತಳ್ಳುವುದು ಸುಲಭ.



ಚಿತ್ರ ೮: Residual block (ಸ್ಲೈಡ್ 41 ರ Figure 2)

ಹೋಲಿಕೆ: ಲೋಕಲ್ ಬಸ್ + ಎಕ್ಸ್‌ಪ್ರೆಸ್ ಮಾರ್ಗ

ಬೆಂಗಳೂರಿನಿಂದ ಮೈಸೂರಿಗೆ ಲೋಕಲ್ ಬಸ್ (ಪ್ರತಿ ನಿಲ್ದಾಣದಲ್ಲಿ ನಿಲ್ಲುತ್ತಾ — ಮಾಹಿತಿ ಕ್ರಮೇಣ ಬದಲಾಗುತ್ತದೆ) ಮತ್ತು ಜೊತೆಗೆ ನಿಲುಗಡೆಯಿಲ್ಲದ ಎಕ್ಸ್‌ಪ್ರೆಸ್ ಮಾರ್ಗ (ಮೂಲ ಮಾಹಿತಿ ನೇರ ತಲುಪುತ್ತದೆ). ಎರಡೂ ಇದ್ದರೆ ಕನಿಷ್ಠ ಮೂಲ ಮಾಹಿತಿ ಕಳೆದುಹೋಗುವುದಿಲ್ಲ.

ಲೆಕ್ಕ ೧: Residual ಜೋಡಣೆ

```
x = [2, 3],  F(x) = [0.1, −0.2]   (ಪದರಗಳು ಕಲಿತ ಚಿಕ್ಕ ತಿದ್ದುಪಡಿ)
ಔಟ್‌ಪುಟ್ = F(x) + x = [2.1, 2.8]
```

ಪದರಗಳಿಗೆ ಏನೂ ಕಲಿಯದಿದ್ದರೂ (F = 0) ಔಟ್‌ಪುಟ್ = x — ಮಾಹಿತಿ ಹಾಗೆಯೇ ಉಳಿಯುತ್ತದೆ.

ಲೆಕ್ಕ ೨: ಗ್ರೇಡಿಯಂಟ್ ಏಕೆ ಕಳೆದುಹೋಗುವುದಿಲ್ಲ? (ಚಿತ್ರಣಾತ್ಮಕ)

ಪ್ರತಿ ಪದರದ ಗ್ರೇಡಿಯಂಟ್ ಗುಣಕ ≈ 0.25 ಎಂದುಕೊಳ್ಳಿ (sigmoid ಯ ಗರಿಷ್ಠದಂತೆ). 10 ಪದರಗಳು:

```
ಸಾಮಾನ್ಯ ನೆಟ್‌ವರ್ಕ್ : 0.25¹⁰        = 0.00000095   (ಬಹುತೇಕ ಸೊನ್ನೆ)
Residual          : (1 + 0.25)¹⁰   ≈ 9.3          (ಶಾರ್ಟ್‌ಕಟ್‌ನ "+1" ದಾರಿಯಿಂದ ಸಂಕೇತ ಉಳಿಯುತ್ತದೆ)
```

d(F(x) + x)/dx = F′(x) + 1 — ಈ "+1" ಗ್ರೇಡಿಯಂಟ್‌ಗೆ ಕುಗ್ಗದ ಒಂದು ನೇರ ದಾರಿ ಕೊಡುತ್ತದೆ. (ಇದು ಕೇವಲ ಕಲ್ಪನಾ ಲೆಕ್ಕ; ನಿಜವಾದ ನೆಟ್‌ವರ್ಕ್‌ನಲ್ಲಿ ಪರಿಣಾಮ ಸಂಕೀರ್ಣ.)

Keras ನಲ್ಲಿ ಚಿಕ್ಕ ResNet (CIFAR-10 ಗೆ, ಸ್ಲೈಡ್ 42 — Functional API)

Residual ಸಂಪರ್ಕಗಳು ಪದರಗಳ ಅನುಕ್ರಮ ಸ್ಟ್ಯಾಕ್ ಅಲ್ಲ (ಕವಲು + ಜೋಡಣೆ), ಆದ್ದರಿಂದ `Sequential` ಬದಲು **Functional API** ಬೇಕು (ಹಿಂದಿನ ಉಪನ್ಯಾಸದಲ್ಲಿ ನೋಡಿದ್ದು).

```
inputs = keras.Input(shape=(32, 32, 3), name="img")       # CIFAR-10: 32×32 ಬಣ್ಣದ ಚಿತ್ರ
x = layers.Conv2D(32, 3, activation="relu")(inputs)       # → 30×30×32
x = layers.Conv2D(64, 3, activation="relu")(x)            # → 28×28×64
block_1_output = layers.MaxPooling2D(3)(x)                # → 9×9×64

x = layers.Conv2D(64, 3, activation="relu", padding="same")(block_1_output)
x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
block_2_output = layers.add([x, block_1_output])          # ← residual: F(x) + x

x = layers.Conv2D(64, 3, activation="relu", padding="same")(block_2_output)
x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
block_3_output = layers.add([x, block_2_output])          # ← ಇನ್ನೊಂದು residual

x = layers.Conv2D(64, 3, activation="relu")(block_3_output)   # → 7×7×64
x = layers.GlobalAveragePooling2D()(x)                    # → 64 (ಪ್ರತಿ ಮ್ಯಾಪ್‌ನ ಸರಾಸರಿ)
x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(10)(x)                             # 10 ವರ್ಗಗಳು

model = keras.Model(inputs, outputs, name="toy_resnet")
model.summary()
```

ಲೆಕ್ಕ (ನನ್ನ ಲೆಕ್ಕ)

```
Conv(32)            : 32 × (3×3×3  + 1) =     896
Conv(64) (32→64)    : 64 × (3×3×32 + 1) =  18,496
ಬ್ಲಾಕ್ 2 ಎರಡು Conv  : 2 × 36,928        =  73,856
ಬ್ಲಾಕ್ 3 ಎರಡು Conv  : 2 × 36,928        =  73,856
ಕೊನೆಯ Conv(64)     : 64 × (9×64 + 1)   =  36,928
Dense(256)          : 64 × 256 + 256    =  16,640
Dense(10)           : 256 × 10 + 10     =   2,570
ಒಟ್ಟು                                 = 223,242   (add ಮತ್ತು pooling: 0)
```

ಆಕಾರ ಪರಿಶೀಲನೆ: MaxPooling2D(3) ನಲ್ಲಿ 28 / 3 = 9.33 → 9 (ಕೆಳಮುಖ ಪೂರ್ಣಾಂಕ). `add` ಗೆ ಎರಡೂ ಇನ್‌ಪುಟ್ ಒಂದೇ ಆಕಾರ (9×9×64) ಇರಬೇಕು — ಅದಕ್ಕೇ ಬ್ಲಾಕ್‌ಗಳಲ್ಲಿ padding="same".

ವಿನ್ಯಾಸಗಳ ಹೋಲಿಕೆ

ವಿನ್ಯಾಸ	ವರ್ಷ	ಮುಖ್ಯ ಕೊಡುಗೆ	ImageNet top-5 ದೋಷ
LeNet-5	1990 ರ ದಶಕ	Conv + Pool ಕಲ್ಪನೆ; ಅಂಕೆ ಗುರುತಿಸುವಿಕೆ	—
AlexNet	2012	ಆಳ CNN + ReLU + Dropout + GPU; ಕ್ರಾಂತಿ	16.4%
VGG16	2014	ಸರಳ 3×3 ಸ್ಟ್ಯಾಕ್, ಆಳ 16 ಪದರ	7.3%
ResNet	2015	ಶಾರ್ಟ್‌ಕಟ್/residual — ನೂರಾರು ಪದರ ಸಾಧ್ಯ	3.6%
ಮಾನವ	—	—	5.0%

ಸ್ಲೈಡ್ 34 ರಲ್ಲಿ 2018 ರವರೆಗಿನ ವಿನ್ಯಾಸಗಳ **ನಿಖರತೆ (top-1) vs ಲೆಕ್ಕಾಚಾರ (GFLOPs = ಒಂದು ಚಿತ್ರಕ್ಕೆ ಬೇಕಾದ ಶತಕೋಟಿ floating-point ಕ್ರಿಯೆಗಳು)** ಹೋಲಿಕೆ ಇದೆ — ಹೆಚ್ಚು ನಿಖರತೆಗೆ ಸಾಮಾನ್ಯವಾಗಿ ಹೆಚ್ಚು ಲೆಕ್ಕಾಚಾರ ಬೇಕು; VGG ಗಳು ಹೆಚ್ಚು GFLOPs ಬಳಸುತ್ತವೆ, ದಕ್ಷ ವಿನ್ಯಾಸಗಳು ಕಡಿಮೆಯಲ್ಲಿ ಹೆಚ್ಚು ನಿಖರತೆ ಪಡೆಯುತ್ತವೆ.

೧೩. ಸಾರಾಂಶ, ಪದಕೋಶ ಮತ್ತು ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

ಸೂತ್ರಗಳ ಸಾರಾಂಶ

ವಿಷಯ	ಸೂತ್ರ	ಉದಾಹರಣೆ
Conv ಔಟ್‌ಪುಟ್ ಗಾತ್ರ	$\lfloor (n + 2p - f)/s \rfloor + 1$	$(28 + 2 - 3)/1 + 1 = 28$
Conv2D ಪ್ಯಾರಾಮೀಟರ್	filters × (f×f×C_in + 1)	64 × (3×3×32 + 1) = 18,496
Conv1D ಪ್ಯಾರಾಮೀಟರ್	filters × (k×C_in + 1)	100 × (5×8 + 1) = 4,100
Dense ಪ್ಯಾರಾಮೀಟರ್	(ಇನ್‌ಪುಟ್ + 1) × ನ್ಯೂರಾನ್	(3136 + 1) × 100 = 313,700
Pooling ಔಟ್‌ಪುಟ್ (2×2, s=2)	n / 2 (ಪ್ರತಿ ಆಯಾಮ)	14 → 7; ಪ್ಯಾರಾಮೀಟರ್ 0
Residual	ಔಟ್‌ಪುಟ್ = F(x) + x	[2,3] + [0.1,−0.2] = [2.1, 2.8]

ಪದಕೋಶ

ಇಂಗ್ಲಿಷ್	ಕನ್ನಡದಲ್ಲಿ ಸರಳ ಅರ್ಥ
Convolution	ಕರ್ನಲ್ ಅನ್ನು ಇನ್‌ಪುಟ್ ಮೇಲೆ ಜಾರಿಸಿ ಗುಣಿಸಿ-ಕೂಡಿಸುವ ಕ್ರಿಯೆ
Kernel / Filter	ಲಕ್ಷಣ ಹುಡುಕುವ ಚಿಕ್ಕ ತೂಕ ಮ್ಯಾಟ್ರಿಕ್ಸ್
Feature map	ಒಂದು ಫಿಲ್ಟರ್‌ನ ಫಲಿತಾಂಶ ನಕ್ಷೆ
Receptive field	ಒಂದು ನ್ಯೂರಾನ್ ನೋಡುವ ಇನ್‌ಪುಟ್ ಪ್ರದೇಶ
Weight sharing	ಒಂದೇ ತೂಕಗಳನ್ನು ಇಡೀ ಚಿತ್ರದಲ್ಲಿ ಮರುಬಳಕೆ
Translation invariance	ವಸ್ತು ಎಲ್ಲೇ ಇದ್ದರೂ ಗುರುತಿಸುವ ಸಾಮರ್ಥ್ಯ
Padding (valid/same)	ಸುತ್ತ ಸೊನ್ನೆ ಸೇರಿಸುವುದು / ಸೇರಿಸದಿರುವುದು
Stride	ಕರ್ನಲ್‌ನ ಹೆಜ್ಜೆ ಗಾತ್ರ
Pooling (max / average)	ಚಿಕ್ಕ ಭಾಗದಿಂದ ಗರಿಷ್ಠ / ಸರಾಸರಿ ಆರಿಸಿ ಗಾತ್ರ ಕುಗ್ಗಿಸುವುದು
Channel	ಚಿತ್ರದ/ಮ್ಯಾಪ್‌ನ ಆಳ (RGB = 3)
Flatten	ಬಹು-ಆಯಾಮದ ಮ್ಯಾಪ್ → ಒಂದು ಸಾಲು
Conv1D	ಅನುಕ್ರಮ (ಪಠ್ಯ/ಸಂಕೇತ) ಮೇಲೆ ಜಾರುವ ಫಿಲ್ಟರ್
Residual / Skip connection	ಇನ್‌ಪುಟ್ ಅನ್ನು ಔಟ್‌ಪುಟ್‌ಗೆ ನೇರ ಸೇರಿಸುವ ಶಾರ್ಟ್‌ಕಟ್
Pre-trained model	ಮೊದಲೇ ದೊಡ್ಡ ದತ್ತಾಂಶದಲ್ಲಿ ತರಬೇತಿಯಾದ ಮಾದರಿ (VGG16 ಇತ್ಯಾದಿ)

ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

1. ಇನ್‌ಪುಟ್ 32×32, ಕರ್ನಲ್ 5×5, padding = 0, stride = 1 ಆದರೆ ಔಟ್‌ಪುಟ್ ಗಾತ್ರ?
2. ಇನ್‌ಪುಟ್ 28×28, ಕರ್ನಲ್ 3×3, padding = 1, stride = 2 ಆದರೆ ಔಟ್‌ಪುಟ್ ಗಾತ್ರ?
3. ಇನ್‌ಪುಟ್‌ನಲ್ಲಿ 8 ಚಾನೆಲ್‌ಗಳಿವೆ. Conv2D(16, (3,3)) ನಲ್ಲಿ ಎಷ್ಟು ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು?
4. [[1, 5], [3, 2]] ಗೆ 2×2 max pooling ಫಲಿತಾಂಶ? 8×8 ಮ್ಯಾಪ್‌ಗೆ 2×2 pooling (stride 2) ನಂತರದ ಗಾತ್ರ?
5. Flatten ನಂತರ ಇನ್‌ಪುಟ್ 5×5×16 ಇದ್ದು Dense(100) ಪದರ ಇದ್ದರೆ ಪ್ಯಾರಾಮೀಟರ್ ಎಷ್ಟು?
6. Conv1D(50, 3) ಅನ್ನು embedding ಆಯಾಮ 16 ಇರುವ ಅನುಕ್ರಮದ ಮೇಲೆ ಬಳಸಿದರೆ ಪ್ಯಾರಾಮೀಟರ್ ಎಷ್ಟು?
7. IMDB CNN ತರಬೇತಿ ಲಾಗ್‌ನಲ್ಲಿ ಯಾವ epoch ನಲ್ಲಿ validation accuracy ಗರಿಷ್ಠ? ಯಾವ ಲಕ್ಷಣ overfitting ತೋರಿಸುತ್ತದೆ?
8. Residual ಬ್ಲಾಕ್‌ನಲ್ಲಿ x = [2, 3], F(x) = [0.5, −1] ಆದರೆ ಔಟ್‌ಪುಟ್ ಎಷ್ಟು?
9. ಪಿಕ್ಸೆಲ್ ಸಾಲು [9, 9, 9, 0, 0] ಗೆ 3 ಅಂಶದ ಸರಾಸರಿ (blur) ಕರ್ನಲ್ ಅನ್ವಯಿಸಿದರೆ ಫಲಿತಾಂಶ?
10. ಒಂದು ಬಣ್ಣದ ಚಿತ್ರದ ಮೇಲೆ Dense ಬದಲು Conv ಬಳಸಿದರೆ ಪ್ಯಾರಾಮೀಟರ್ ಏಕೆ ಕಡಿಮೆ?

ಉತ್ತರಗಳು: (೧) 32 − 5 + 1 = **28×28**. (೨) ⌊(28 + 2 − 3)/2⌋ + 1 = ⌊13.5⌋ + 1 = **14×14**. (೩) 16 × (3×3×8 + 1) = 16 × 73 = **1,168**. (೪) **5**; 8×8 → **4×4**. (೫) (400 + 1) × 100 = **40,100**. (೬) 50 × (3×16 + 1) = 50 × 49 = **2,450**. (೭) **epoch 3** (0.8764); training accuracy ಏರುತ್ತಿದ್ದು validation accuracy/loss ಕೆಟ್ಟಾಗುವುದು (epoch 4–5 ರಲ್ಲಿ val_loss 0.3603, 0.4517 ಏರಿಕೆ). (೮) [2.5, 2.0]. (೯) **[9, 6, 3]**. (೧೦) ಒಂದೇ ಚಿಕ್ಕ ಫಿಲ್ಟರ್ ಇಡೀ ಚಿತ್ರದ ಮೇಲೆ ಮರುಬಳಕೆಯಾಗುತ್ತದೆ (weight sharing) ಮತ್ತು ಪ್ರತಿ ನ್ಯೂರಾನ್ ಕೇವಲ ಸ್ಥಳೀಯ ಭಾಗವನ್ನು ನೋಡುತ್ತದೆ; Dense ನಲ್ಲಿ ಪ್ರತಿ ನ್ಯೂರಾನ್‌ಗೆ ಎಲ್ಲ ಪಿಕ್ಸೆಲ್‌ಗಳಿಗೂ ಪ್ರತ್ಯೇಕ ತೂಕ ಬೇಕು.